



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第7章 存储系统

2026年春季学期

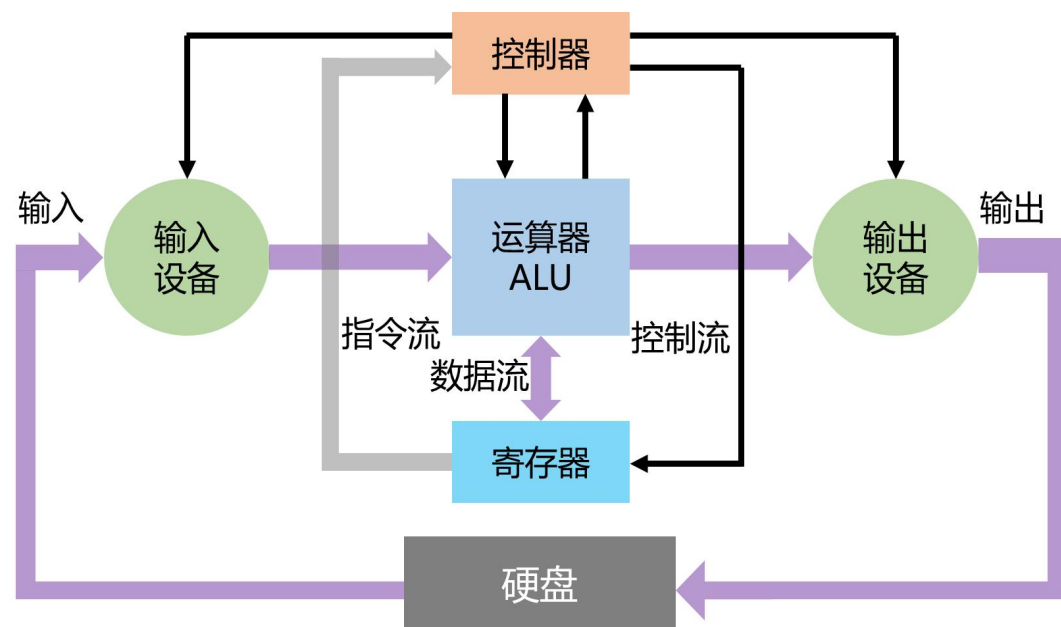
上一讲内容

第 2 页

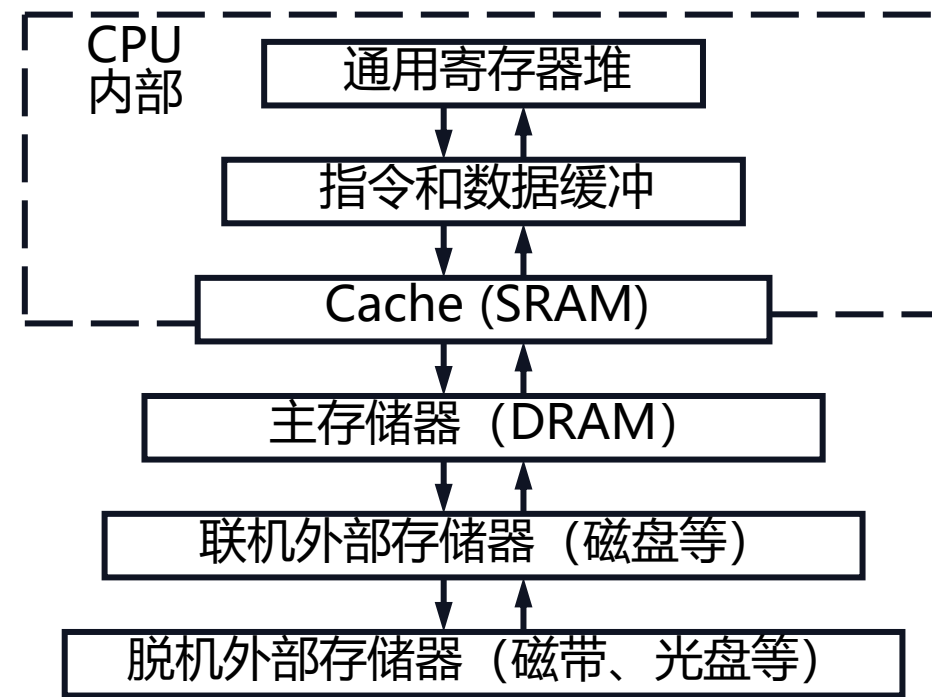
瓶颈问题：需要访问外部存储器（取指令和数据），CPU执行时间花在存储器访问

主要原因：寄存器和外部存储器的容量与访问速度差在，且越来越大

解决方法：采用层次化存储系统的设计与管理



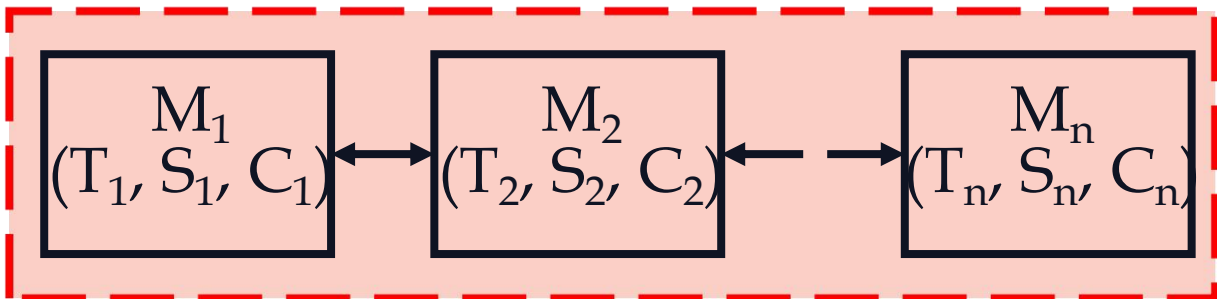
每
存
储
的
容
量
价
格
越
来
越
大
便
宜



访
问
速
度
越
来
越
快

上一讲内容

第 3 页



M_i ($i=1, 2, \dots$) 表示第 i 层存储器

T_i ($i=1, 2, \dots$) 表示第 i 层存储器的访问时间

S_i ($i=1, 2, \dots$) 表示第 i 层存储器的容量

C_i ($i=1, 2, \dots$) 表示第 i 层存储器的总价格

程序
访问
的局
部性

- 顺序局部性 (Order Locality) : 在典型程序中, 除转移类指令外, 大部分指令是顺序进行的。
- 时间局部性 (Temporal Locality) : 一个数据或指令被访问, 近期内它被再次访问的概率很大, 例如程序循环等。
- 空间局部性 (Spatial Locality) : 一个数据被访问, 近期内它附近的数据被再次访问的概率很大, 例如数组的连续存储

$$T = H \cdot T_1 + (1 - H) \cdot T_2$$

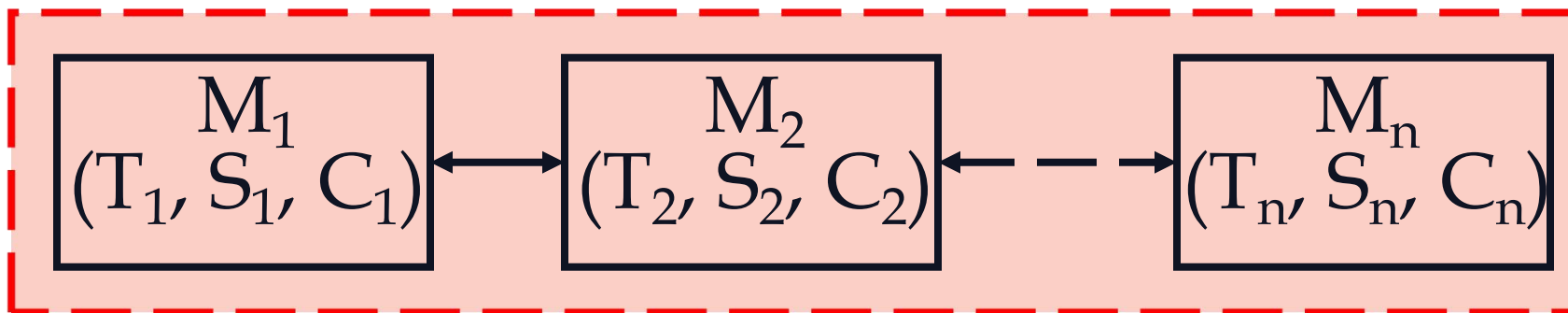
$$e = \frac{T_1}{T} = \frac{T_1}{H \cdot T_1 + (1 - H) \cdot T_2}$$

假设: M_1 是 M_2 的真子集; 所有的数据都能在 M_2 中找到。

访问方式: 没有在 M_1 中找到的数据则访问 M_2 , 再把数据提供给处理器 **(无预取)**

上一讲内容

第 4 页



$$T \approx \min(T_1, T_2, \dots, T_n) \quad S \approx \max(S_1, S_2, \dots, S_n) \quad C \approx \min(C_1, C_2, \dots, C_n)$$

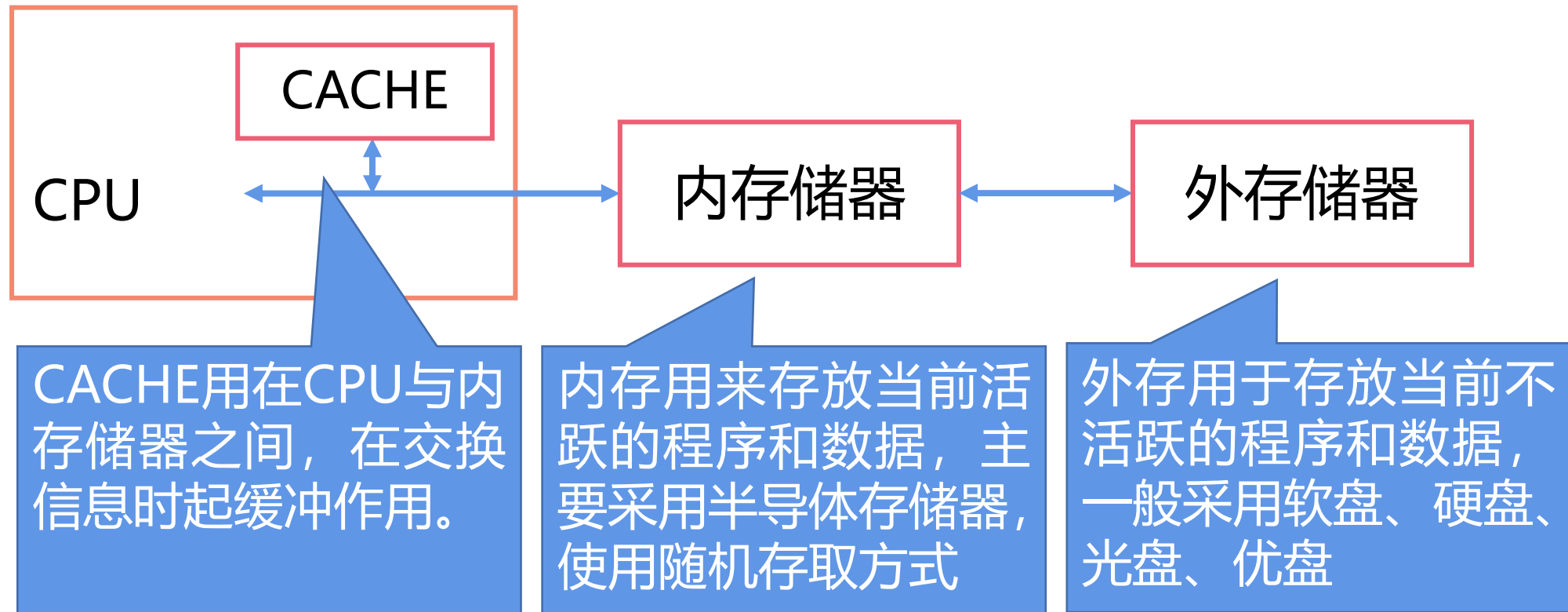
设计方法：（1）相临两级存储器的访问速度与存储容量建议~10倍；（2）相临两级存储器的预取数据量不宜过大（256B~1024B）

技术延展：由里向外逐2层计算多层存储系统的平均访问时间 T 和访问效率 e

上一讲内容

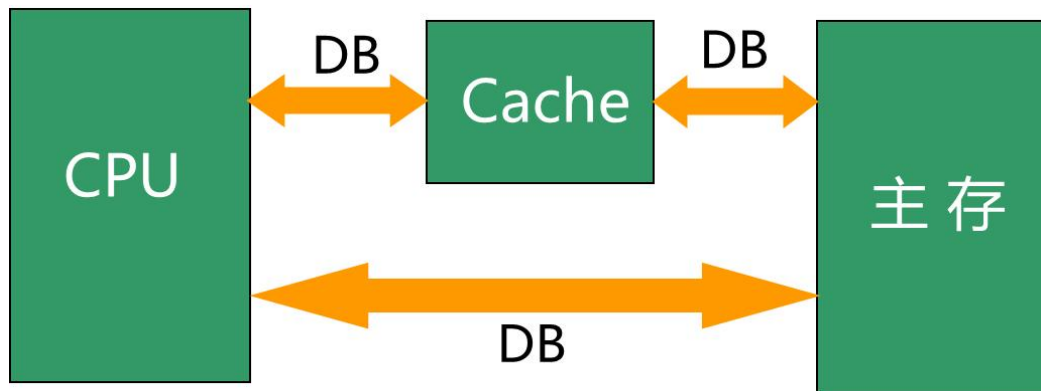
第 5 页

现代计算机的存储系统是通过软、硬件结合，形成了CACHE—内存储器—外存储器组合的典型的**三层存储系统**



上一讲内容

第 6 页



Cache是按块进行管理的。Cache和主存均被分割成大小相同的块。信息以块为单位调入Cache

映象规则： M_2 存储器中一个块调入 M_1 存储器时，可以放在哪些位置上

具体方案：直接映射 ($m \rightarrow 1$)、全相联映射 ($m \rightarrow 1$)、组相联映射 ($m/n \rightarrow 1$)

查找算法：当所要访问的块在高一层存储器中时，如何找到该块？

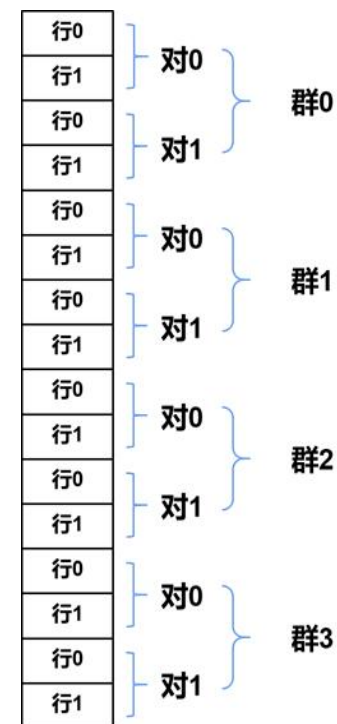
具体方案：并行查找、串行查找（前瞻预测）

替换算法：当发生不命中时，应替换哪一块？

具体方案：随机法、FIFO、LRU等（硬件实现：层次法）

写策略：当进行写访问时，应进行哪些操作？

具体方案：命中（写直达、写回）、缺失（按写分配法、不按写分配法）



上一讲内容

第 7 页

$CPU\text{时间} = (\text{CPU执行周期数} + \text{访存次数} \times \text{不命中率} \times \text{不命中开销}) \times \text{时钟周期时间}$

$$\begin{aligned} CPU\text{时间} &= IC \times \left(CPI_{\text{execution}} + \frac{\text{访存次数}}{\text{指令数}} \times \text{不命中率} \times \text{不命中开销} \right) \times \text{时钟周期时间} \\ &= IC \times (CPI_{\text{execution}} + \text{每条指令的平均访存次数} \times \text{不命中率} \\ &\quad \times \text{不命中开销}) \times \text{时钟周期时间} \end{aligned}$$

一点声明:

不命中率=缺失率

不命中开销=缺失开销

1. 平均访存时间 = 命中时间 + 不命中率 × 不命中开销

2. 可以从三个方面改进Cache的性能：

- 降低不命中率
- 减少不命中开销
- 减少Cache命中时间

3. 下面介绍3类Cache优化技术

- 降低不命中率
- 减少不命中开销
- 减少命中时间

7.5 减少命中时间

7.5.1 容量小、结构简单的Cache

第 10 页

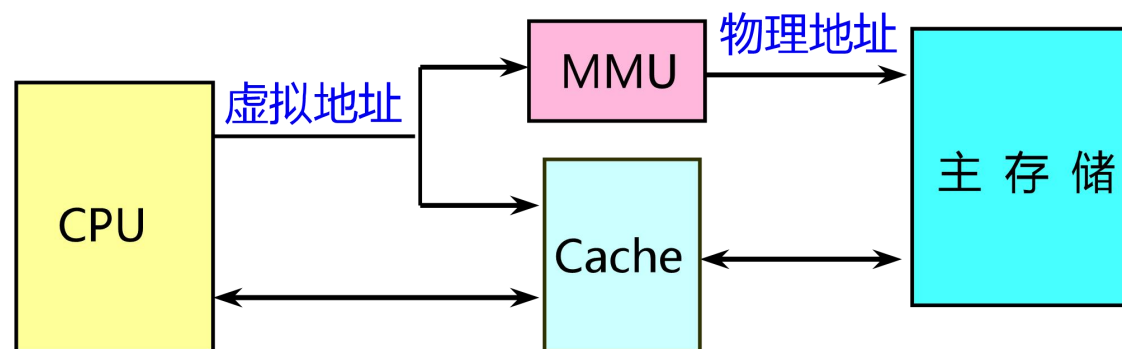
1. 硬件越简单，速度就越快；
2. 应使Cache足够小，以便可以与CPU一起放在同一块芯片上。

某些设计采用了一种折衷方案：

把Cache的标识放在片内，而把Cache的数据存储器放在片外。

3. 虚拟Cache

- 可以直接用虚拟地址进行访问的Cache。标识存储器中存放的是虚拟地址，进行地址检测用的也是虚拟地址。
- 优点：
 - 在命中时不需要地址转换，省去了地址转换的时间。即使不命中，地址转换和访问Cache也是并行进行的，其速度比物理Cache快很多。



虚拟Cache存储系统

7.5.2 虚拟Cache

第 12 页

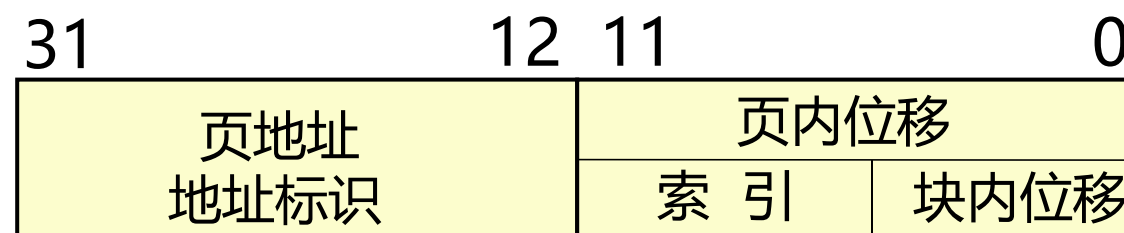
4. 虚拟索引 + 物理标识

- 优点：兼得虚拟Cache和物理Cache的好处
- 局限性：Cache容量受到限制
(页内位移)

$$\text{Cache容量} \leq \text{页大小} \times \text{相联度}$$

5. 举例：IBM3033的Cache

- 页大小 = 4KB 相联度 = 16
- Cache容量 = $16 \times 4\text{KB} = 64\text{KB}$



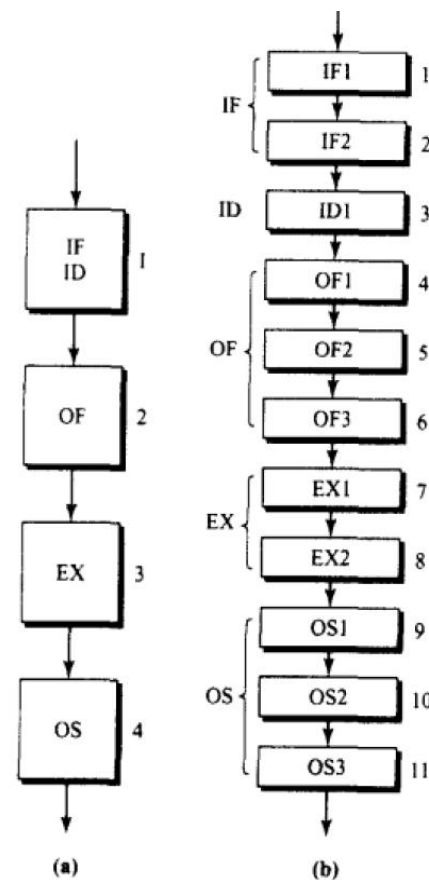
7.5.3 Cache访问流水化

第 13 页

1. 对第一级Cache的访问按流水方式组织
2. 访问Cache需要多个时钟周期才可以完成

例如

- ❑ Pentium访问指令Cache需要一个时钟周期
- ❑ Pentium Pro到Pentium III需要两个时钟周期
- ❑ Pentium 4 则需要4个时钟周期



(a) 一个4级指令流水线的例子；(b) 一个11级指令流水线的例子

7.5.4 踪迹Cache

第 14 页

1. 开发指令级并行性所遇到的一个挑战是：

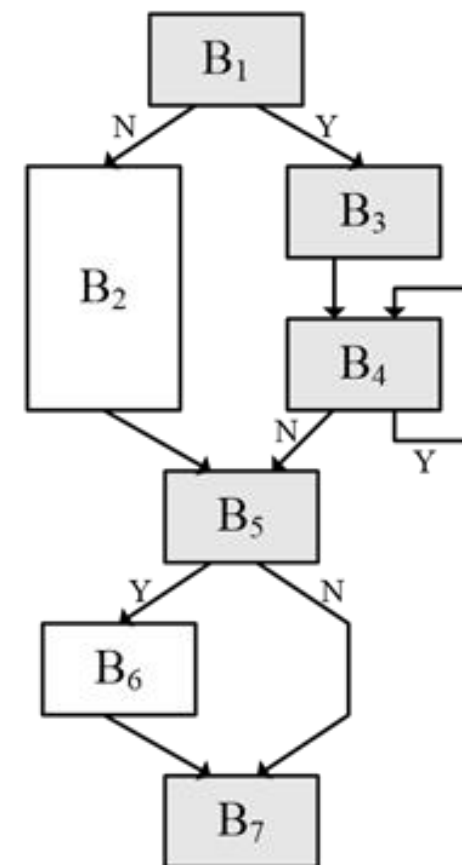
当要每个时钟周期流出超过4条指令时，要提供足够多条彼此互不相关的指令是很困难的。

2. 一个解决方法：采用踪迹 Cache

存放CPU所执行的动态指令序列，包含了由分支预测展开的指令，该分支预测是否正确需要在取到该指令时进行确认。

3. 优缺点

- 地址映象机制复杂，
- 相同的指令序列有可能被当作条件分支的不同选择而重复存放，
- 能够提高指令Cache的空间利用率



7.5.5 Cache优化技术总结

第 15 页

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说明
增加块大小	+	—		0	实现容易； Pentium 4 的第二级Cache采用了128字节的块
增加Cache容量	+			1	被广泛采用，特别是第二级Cache
提高相联度	+		—	1	被广泛采用
牺牲Cache	+			2	AMD Athlon 采用了 8 个 项 的 Victim Cache
伪相联Cache	+			2	MIPS R10000的第二级Cache采用

“+”号：表示改进了相应指标。

空格栏：表示它对该指标无影响。

“—”号：表示它使该指标变差。

复杂性：0表示最容易，3表示最复杂。

7.5.5 Cache优化技术总结

第 16 页

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说明
编译器控制的预取	+			3	需同时采用非阻塞Cache；有几种微CPU提供了对这种预取的支持
用编译技术减少Cache不命中次数	+			0	向软件提出了新要求；有些机器提供了编译器选项
使读不命中优先于写		+	-	1	在单处理机上实现容易，被广泛采用
写缓冲归并		+		1	与写直达合用，广泛应用，例如21164，UltraSPARC III
尽早重启动和关键字优先		+		2	被广泛采用
非阻塞Cache		+		3	在支持乱序执行的CPU中使用

“+”号：表示改进了相应指标。

空格栏：表示它对该指标无影响。

“-”号：表示它使该指标变差。

复杂性：0表示最容易，3表示最复杂。

7.5.5 Cache优化技术总结

第 17 页

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说明
两级Cache			+	2	硬件代价大；两级Cache的块大小不同时实现困难；被广泛采用
容量小且结构简单的Cache	—		+	0	实现容易，被广泛采用
对Cache进行索引时不必进行地址变换			+	2	对于小容量Cache来说实现容易，已被Alpha 21164和UltraSPARC III采用
流水化Cache访问			+	1	被广泛采用
Trace Cache			+	3	Pentium 4 采用

“+”号：表示改进了相应指标。

空格栏：表示它对该指标无影响。

“-”号：表示它使该指标变差。

复杂性：0表示最容易，3表示最复杂。

7.6 并行主存系统

7.6 并行主存系统

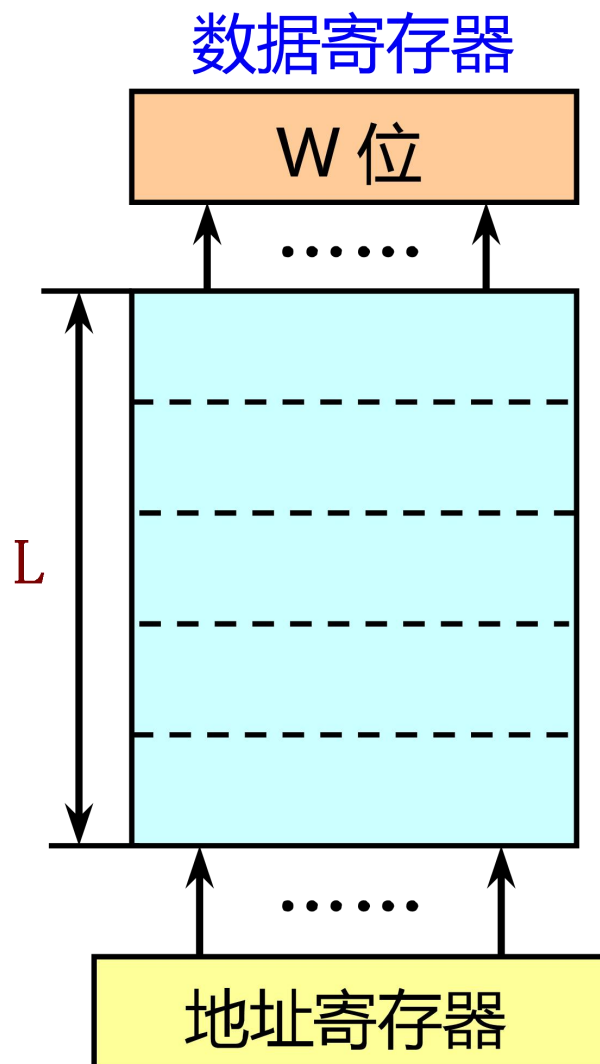
第 19 页

■ 一个单体单字宽的存储器

- 字长与CPU的字长相同。
- 每一次只能访问一个存储字。
假设该存储器的访问周期是 T_M ，
字长为 W 位，则其带宽为：

$$B_M = \frac{W}{T_M}$$

单片存储器容量有限



7.6 并行主存系统

第 20 页

单个RAM芯片的容量往往较小，要组成一定容量和一定字长的主存储器，必须用多个芯片进行有机地组合。

存储容量： $S = M \times W \times B$

M—模块数，W—每个模块的字(或单元)数，B—字长

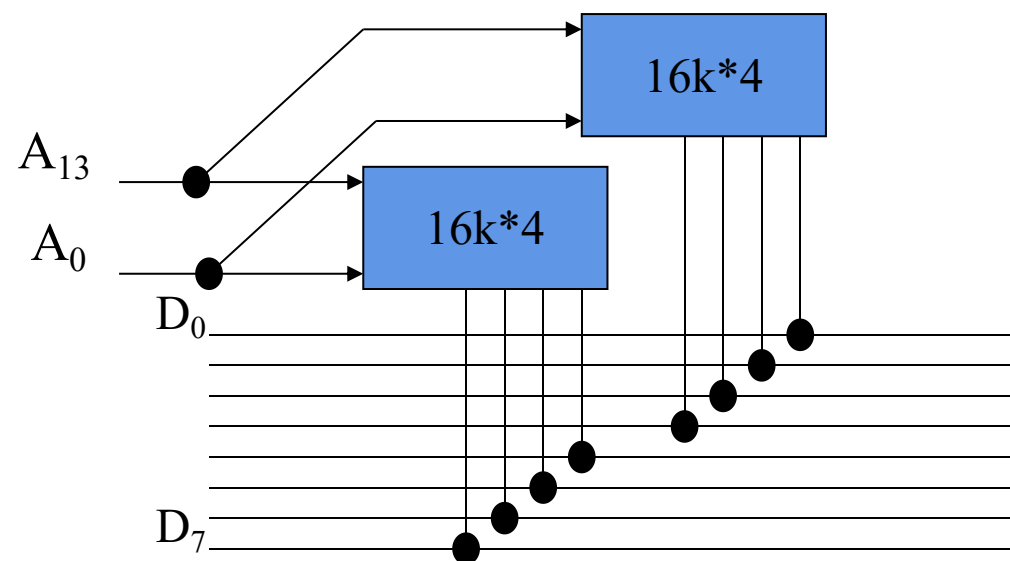
设主存的容量为：W字×b位，芯片的容量为：Ws字×bs位

(1) 位扩展 $W = W_s$, $b > b_s$ 。

例：用16k×4位的芯片组成16k×8位的存储器。

$$W = W_s = 16k \quad b = 2b_s$$

(共需2片)



7.6 并行主存系统

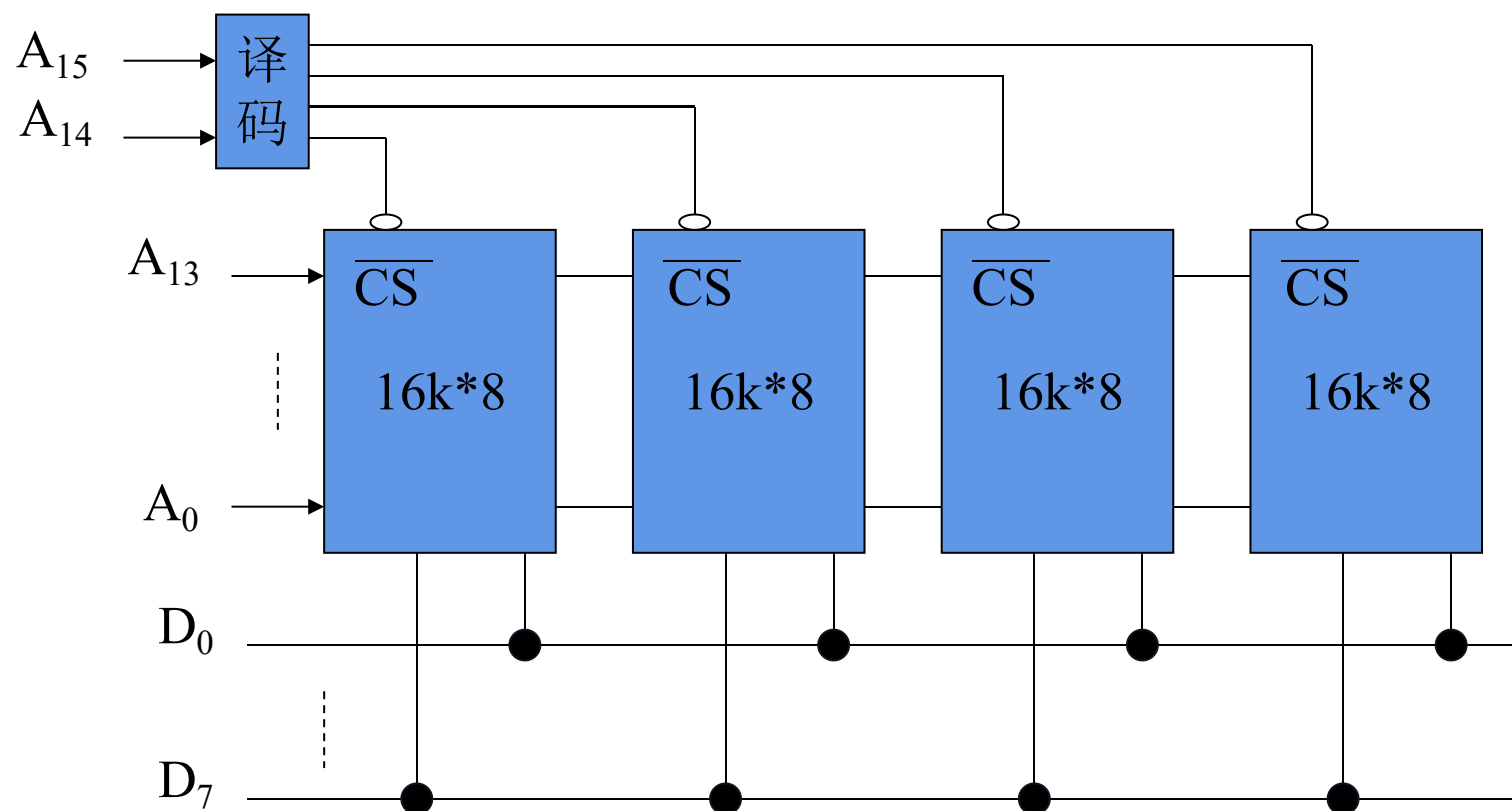
第 21 页

(2) 字扩展

$W > W_s$, $b = b_s$ 。

例：用16k×8位的芯片组成64k×8位的存储器。

$W = 4W_s$
 $b = b_s = 8$
(共需4片)



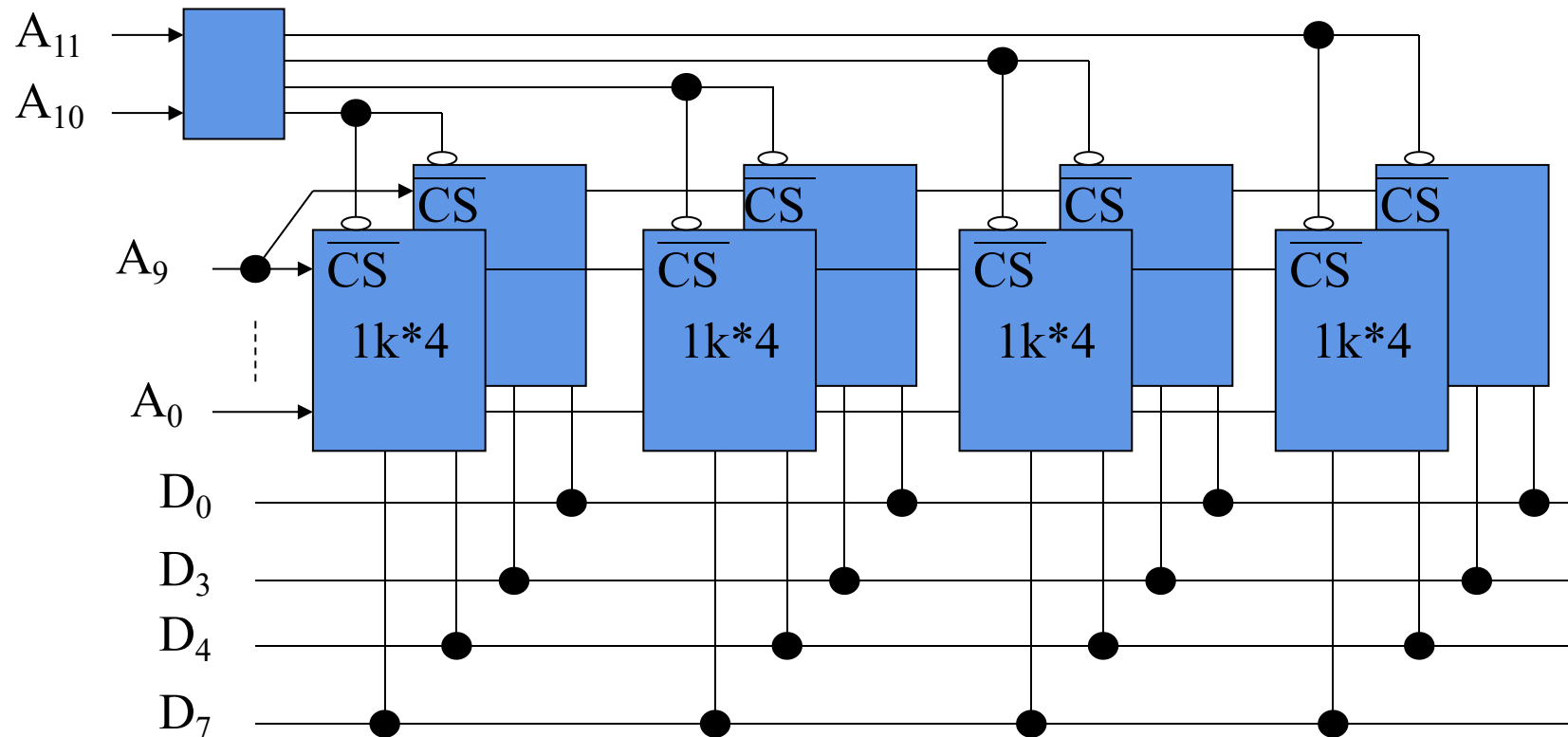
7.6 并行主存系统

第 22 页

(3) 字位扩展

$W > W_s$, $b > b_s$, 共需 $(W / W_s) \times (b / b_s)$ 片

例：用1k×4位的芯片组成4k×8位的存储器。



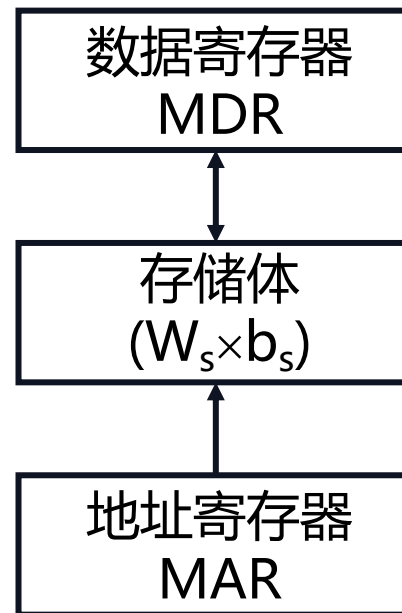
7.6 并行主存系统

第 23 页

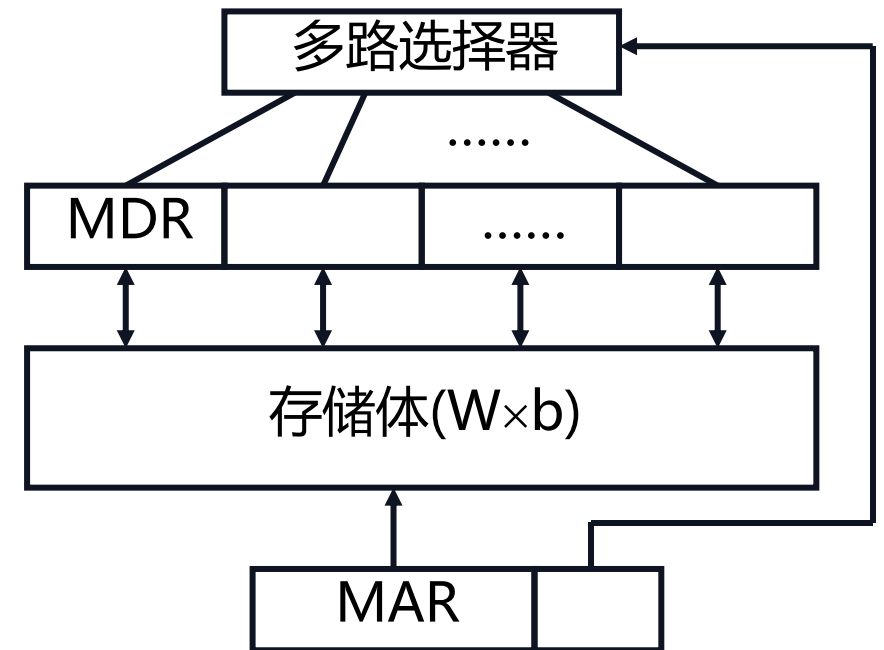
■ 在相同的器件条件（即 T_M 相同）下，可以采用两种并行存储器结构来提高主存的带宽：

■ 两种编址方法：

- 单体多字存储器
- 多体交叉存储器



一般存储器



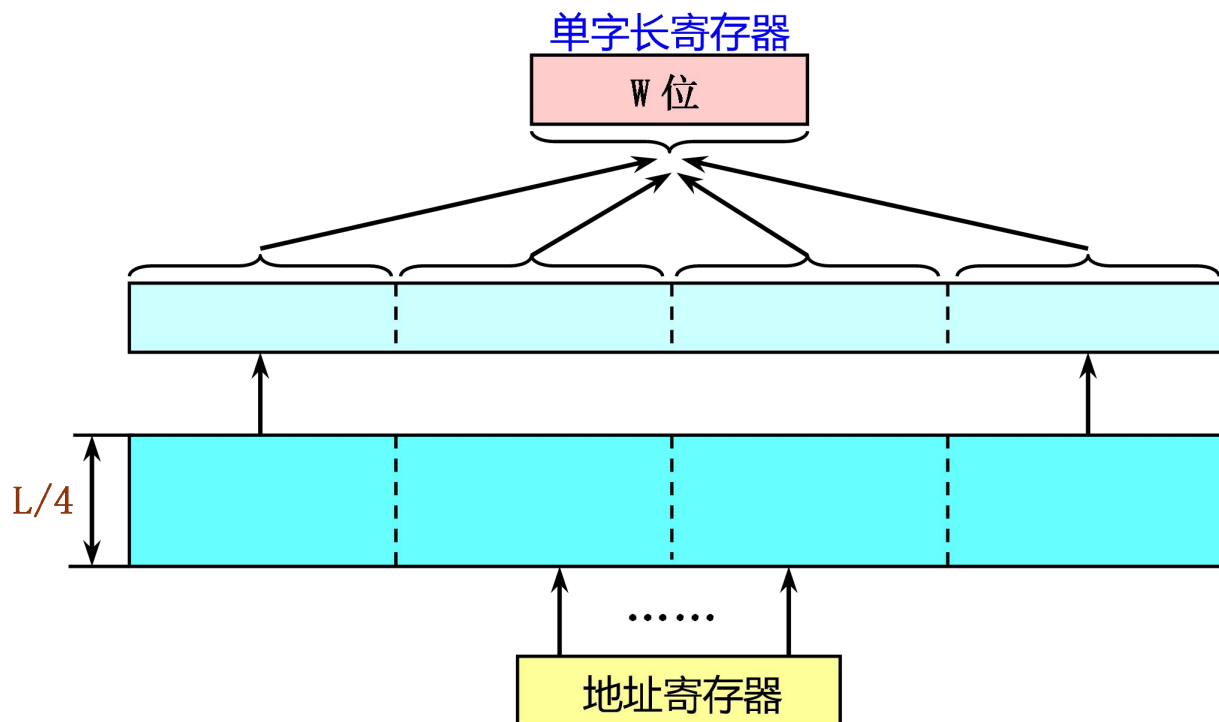
并行访问存储器

例如，共需芯片 $(W / W_s) * (b / b_s) = 8$ 片，分为4个组，每组2片。地址 $A_{11}A_{10}$ 译码后选中一个组，再由 $A_9 \sim A_0$ 选中组内某个单元，进行读/写操作。

7.6.1 单体多字存储器

第 24 页

1. 单体多字存储器：一个单体 m 字（这里 $m=4$ ）存储器



$$B_M = m \times \frac{W}{T_M}$$

- 存储器能够每个存储周期读出 m 个CPU字。因此其最大带宽提高到原来的 m 倍。
- 单体多字存储器的实际带宽比最大带宽小

7.6.1 单体多字存储器

第 25 页

2. 优缺点

- **优点**：实现简单
- **缺点**：访存效率不高

原因：

- 如果一次读取的m个指令字中有分支指令，而且分支成功，那么该分支指令之后的指令是无用的。
- 一次取出的m个数据不一定都是有用的。另一方面，当前执行指令所需要的多个操作数也不一定正好都存放在同一个长存储字中。
- 写入有可能变得复杂。
- 当要读出的数据字和要写入的数据字处于同一个长存储字内时，读和写的操作就无法在同一个存储周期内完成。

1	2	3 (读取)	4
5	6	7	8 (写入)
9 (写入)	10 (写入)	11	12
13	14 (更新)	15	16

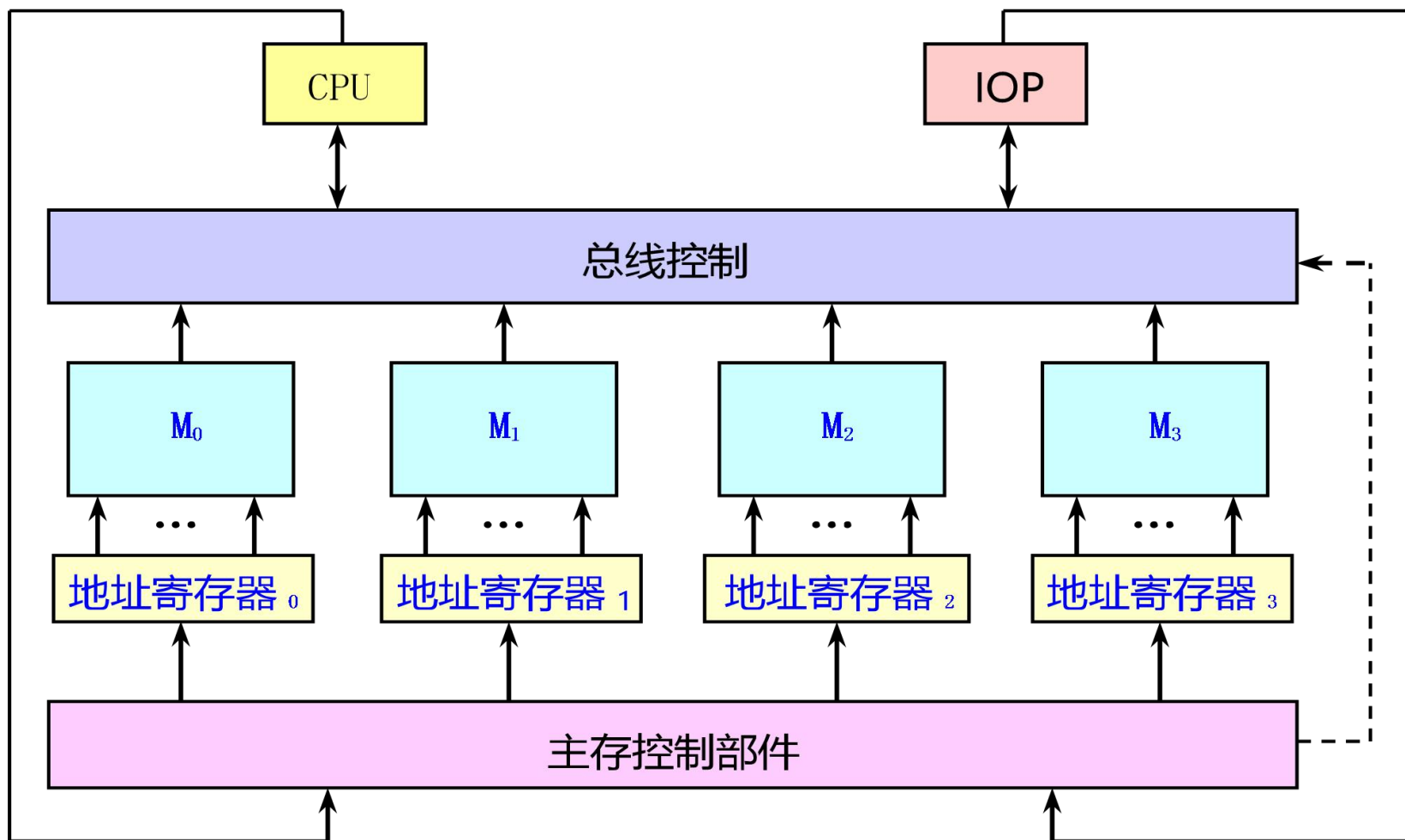
7.6.2 多体交叉存储器

第 26 页

1. 多体交叉存储器：由多个单字存储体构成，每个体都有自己的地址寄存器以及地址译码和读/写驱动等电路。
2. 问题：对多体存储器如何进行编址？
 - 存储器是按顺序线性编址的。如何在二维矩阵和线性地址之间建立对应关系？
 - 两种编址方法
 - 高位交叉编址
 - 低位交叉编址 （有效地解决访问冲突问题）

7.6.2 多体交叉存储器

第 27 页



多体 ($m=4$) 交叉存储器

- 对存储单元矩阵按列优先的方式进行编址
- 特点：同一个体中的高 $\log_2 m$ 位都是相同的（体号）



7.6.2 多体交叉存储器

第 29 页

- 处于第*i*行第*j*列的单元，即体号为*j*、体内地址为*i*的单元，其线性地址为：

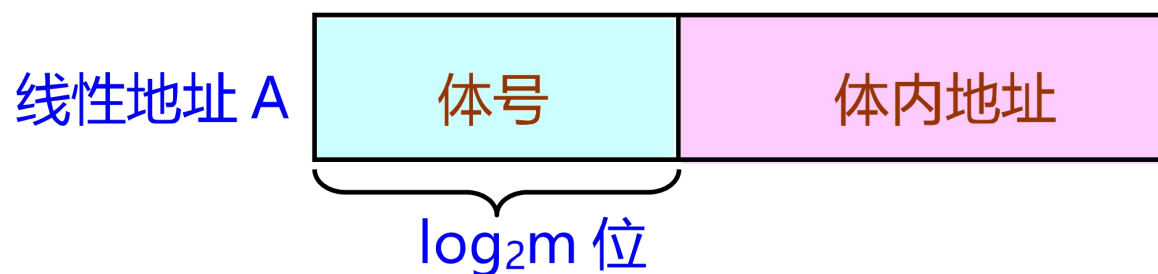
$$A = j \times n + i$$

其中： $j = 0, 1, 2, \dots, m - 1$ ； $i = 0, 1, 2, \dots, n - 1$

- 一个单元的线性地址为*A*，则其体号*j*和体内地址*i*为：

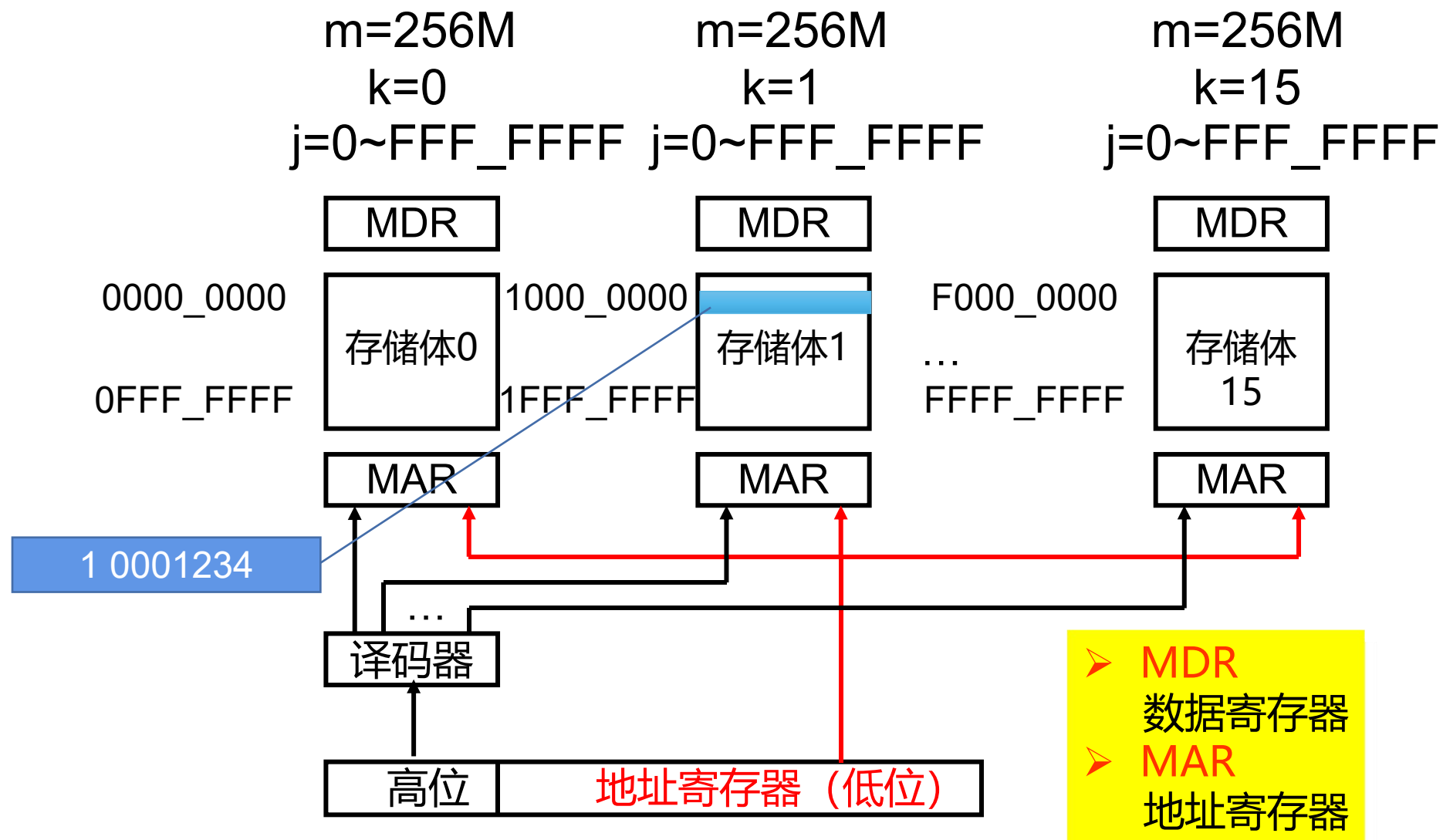
$$j = \left\lfloor \frac{A}{n} \right\rfloor \quad i = A \bmod n$$

- 把*A*表示为二进制数，则其高 $\log_2 m$ 位就是体号，而剩下的部分是体内地址。



7.6.2 多体交叉存储器

第 30 页



■ 目的

扩大存储器容量。

■ 例子

目前，大部分计算机系统中所采用的模块化主存储器通常都是采用高位交叉编址方法实现。

■ 应用

在单任务系统中：可用于扩大存储器容量，且扩充性好。

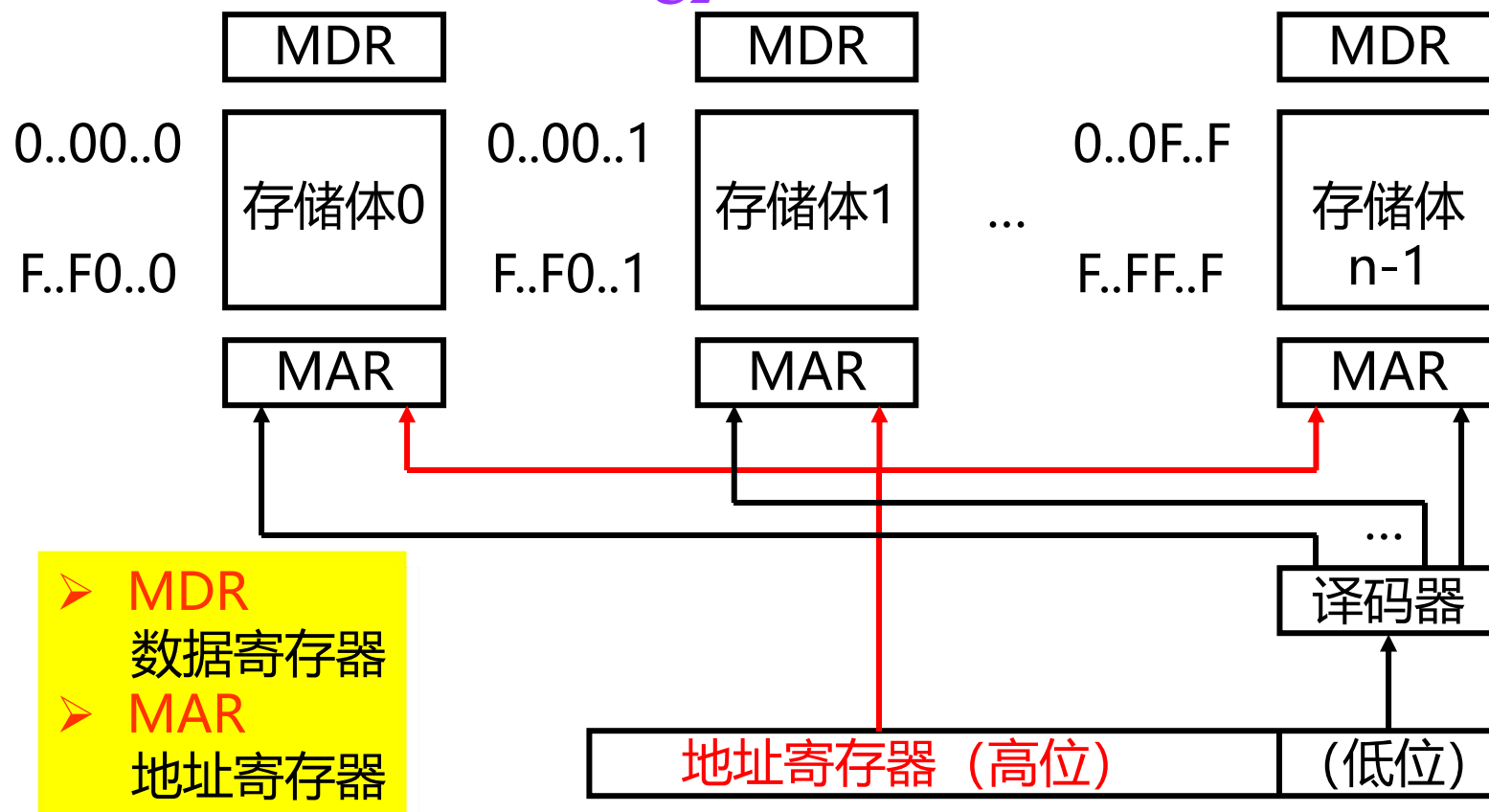
在多任务或多用户系统中：可以通过把不同的任务分配给不同的存储体完成来提高存储器的访问速度。

7.6.2 多体交叉存储器

第 32 页

4. 低位交叉编址

- 对存储单元矩阵按行优先进行编址
- **特点：**同一个体中的低 $\log_2 m$ 位都是相同的体号



7.6.2 多体交叉存储器

第 33 页

- 处于第*i*行第*j*列的单元，即体号为*j*、体内地址为*i*的单元，其线性地址为：

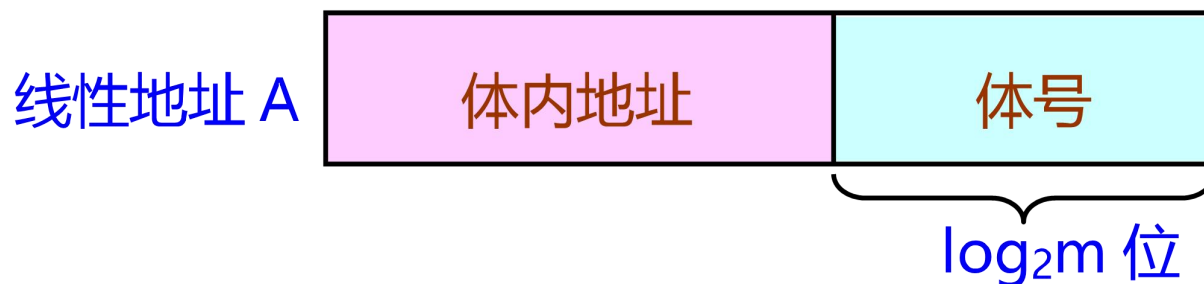
$$A = i \times m + j$$

其中： $i = 0, 1, 2, \dots, n - 1$; $j = 0, 1, 2, \dots, m - 1$

- 一个单元的线性地址为*A*，则其体号*j*和体内地址*i*为：

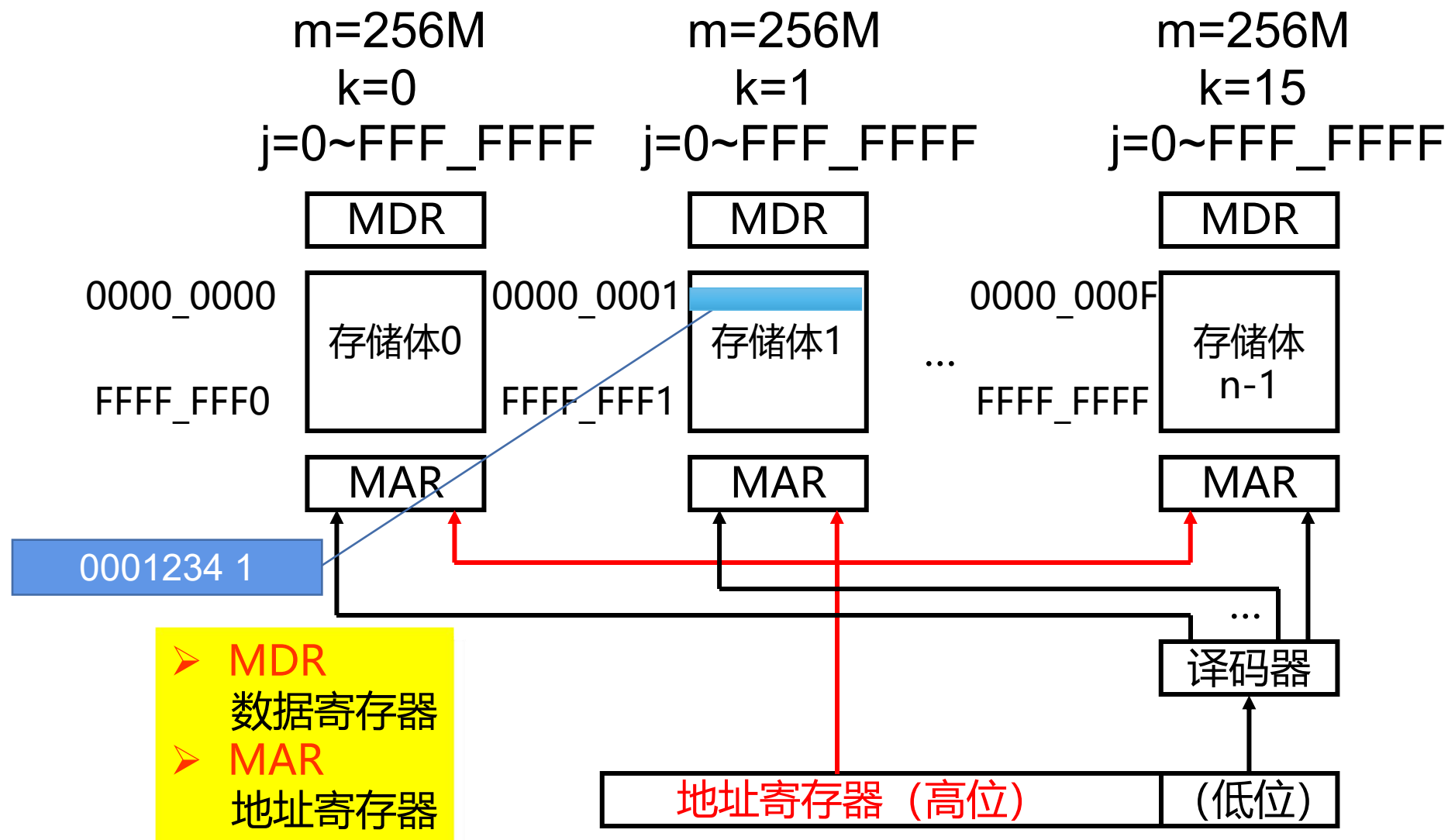
$$i = \left\lfloor \frac{A}{m} \right\rfloor \quad j = A \bmod m$$

- 把*A*表示为二进制数，则其低 $\log_2 m$ 位就是体号，而剩下的部分就是体内地址



7.6.2 多体交叉存储器

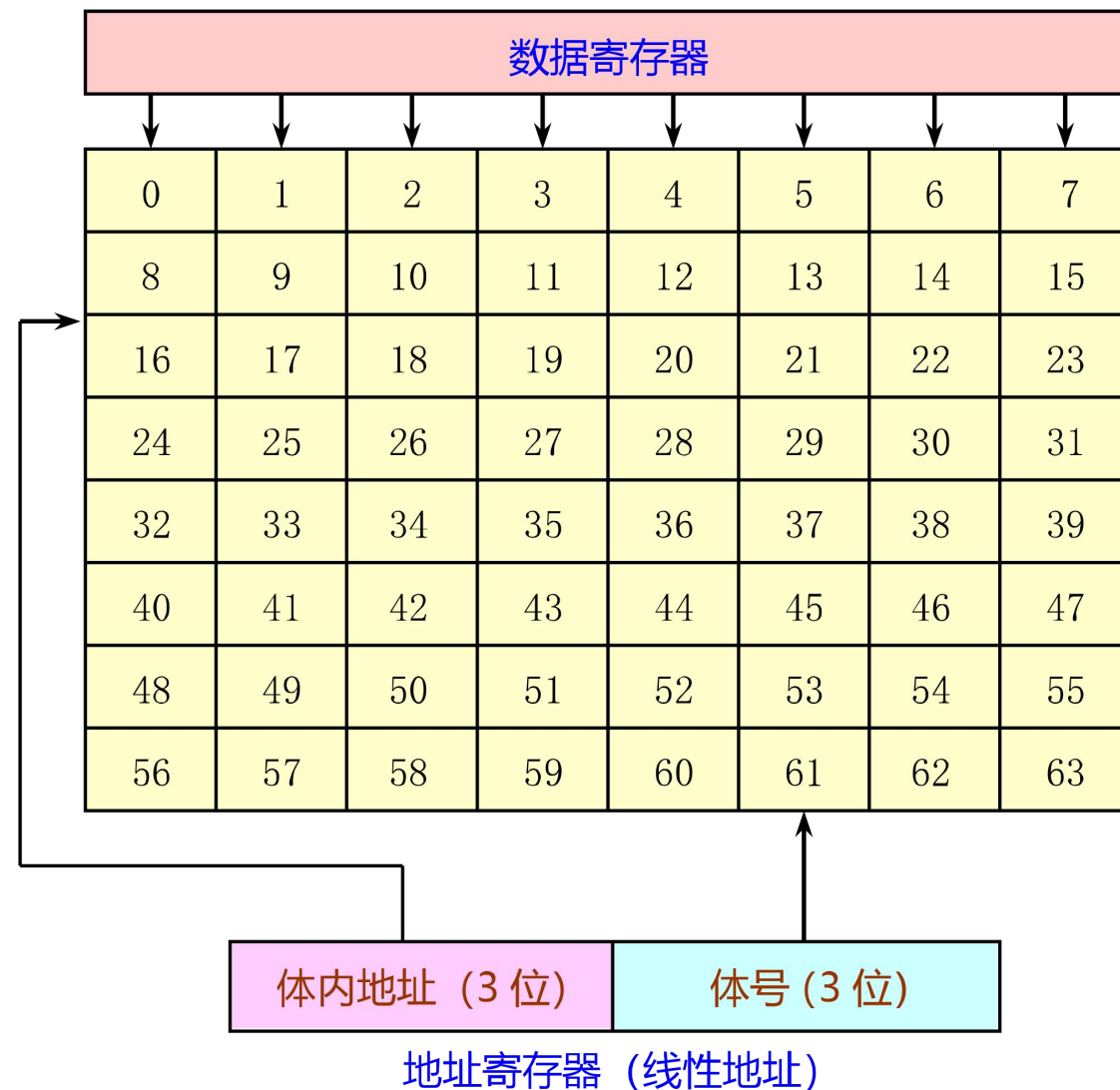
第 34 页



7.6.2 多体交叉存储器

第 35 页

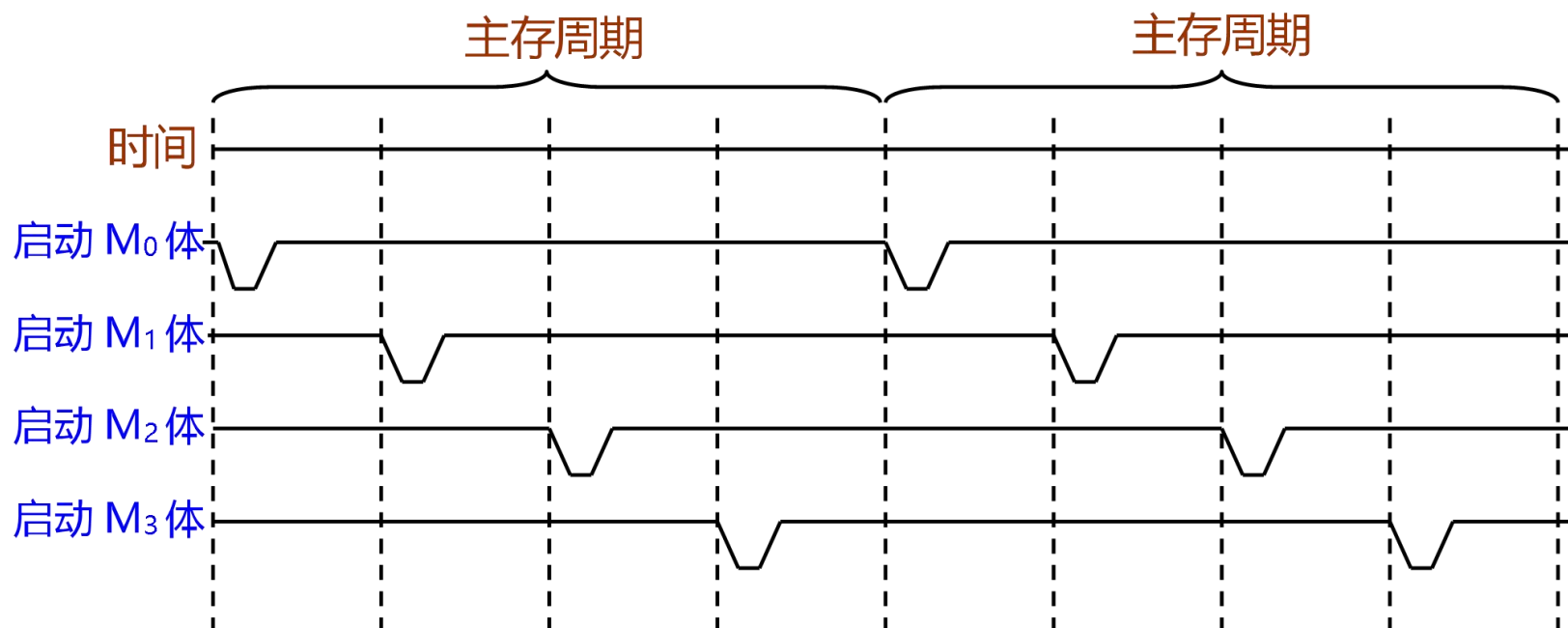
例7.12 采用低位交叉编址的存储器
由8个存储体构成、总容量为64。格子中的
编号为线性地址。



7.6.2 多体交叉存储器

第 36 页

- 为了提高主存的带宽，需要多个或所有存储体能并行工作。
 - 在每一个存储周期内，分时启动 m 个存储体。
 - 如果每个存储体的访问周期是 T_M ，则各存储体的启动间隔为： $t=T_M/m$ 。
 - 增加 m 的值就能够提高主存储器的带宽。但是由于存在访问冲突，实际加速比小于 m 。



- 目的
提高存储器访问速度。
- 实现
为此在一个存储周期内， n 个存储体必须同时或分时启动，实际上是一种采用流水线方式工作的并行存储器。

7.6.2 多体交叉存储器

第 37 页

5. 通过一个模型分析并行主存系统的实际带宽

- 一个由m个独立分体组成的主存系统
- CPU发出的一串地址为 $A_1, A_2, \dots, A_q$ 的访存申请队列
- 存储控制器扫描这个队列，并截取从头起的 $A_1, A_2, \dots, A_k$ 序列作为申请序列。
 - 申请序列是满足以下条件的最长序列：k个地址所访问的存储器单元都处在不同的分体中。
 - $A_1 \sim A_k$ 不一定是顺序地址，只要它们之间不出现分体冲突。
 - k越接近于m，系统的效率就越高。
- 设 $P(k)$ 表示申请序列长度为k的概率，用B表示k的平均值，则

$$B = \sum_{k=1}^m k \cdot P(k) \quad \text{其中：} k=1, 2, \dots, m$$

- 每个主存周期所能访问到的字数的平均值，正比于主存实际带宽。

7.6.2 多体交叉存储器

第 38 页

■ $P(k)$ 与具体程序的运行状况密切相关。如果访存申请队列都是指令的话, 那么影响最大的是转移概率 λ 。

■ 转移概率 λ : 给定指令的下条指令地址为非顺序地址的概率。

➤ 当 $k = 1$ 时, 所表示的情况是: 第一条就是转移指令且转移成功。

$$P(1) = \lambda = (1 - \lambda)^0 \cdot \lambda$$

➤ 当 $k = 2$ 时, 所表示的情况是: 第一条指令没有转移 (其概率为 $1 - \lambda$), 第二条是转移指令且转移成功。所以有:

$$P(2) = (1 - \lambda)^1 \cdot \lambda$$

➤ 同理, $P(3) = (1 - \lambda)^2 \cdot \lambda$

➤ 依此类推, $P(k) = (1 - \lambda)^{k-1} \cdot \lambda$ 其中: $1 \leq k < m$

7.6.2 多体交叉存储器

第 39 页

- 如果前 $m - 1$ 条指令均不转移，则不管第 m 条指令是否转移， k 都等于 m ，因此有：

$$P(m) = (1 - \lambda)^{m-1}$$

$$B = \sum_{k=1}^m k \cdot P(k) = 1 \cdot \lambda + 2 \cdot (1 - \lambda) \cdot \lambda + 3 \cdot (1 - \lambda)^2 \cdot \lambda + \cdots + (m - 1)(1 - \lambda)^{m-2} \cdot \lambda + m(1 - \lambda)^{m-1}$$

$$B = \sum_{i=0}^{m-1} (1 - \lambda)^i$$

$$B = \frac{1 - (1 - \lambda)^m}{\lambda}$$

- 若每条指令都是转移指令且转移成功（ $\lambda=1$ ），则 $B=1$ ，表明在此种情况下使用并行多体交叉存取的实际带宽和使用单体单字的一样；
- 若每条指令都转移不成功（ $\lambda=0$ ），则在此种情况下使用并行多体交叉存取的效率最高。

7.6.2 多体交叉存储器

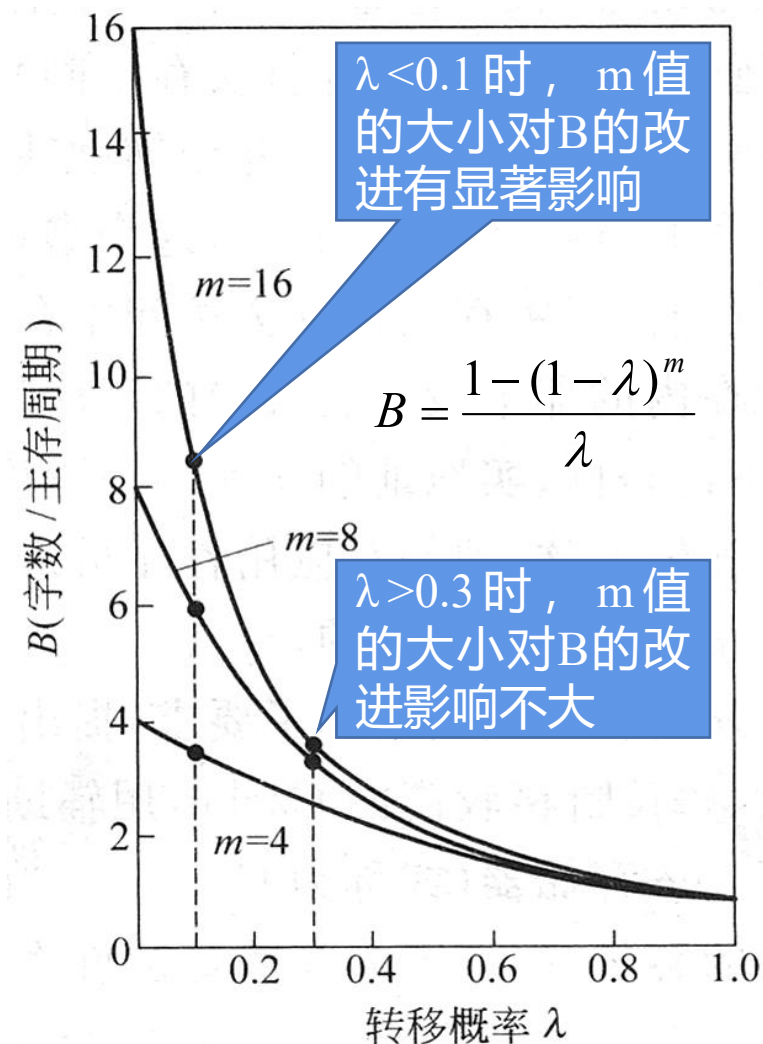
第 40 页

■ m 等于4、8、16时， B 与 λ 的关系曲线

- 对于数据来说，由于其顺序性差， m 值的增大给 B 带来的好处就更差一些。
- 若机器主要是运行标量运算的程序，一般取 $m \leq 8$ 。
- 如果是向量处理机，其 m 值可以取大些。

■ 单纯靠增大 m 来提高并行主存系统的带宽是有限的，而且性能价格比还会随 m 的增大而下降。

- 原因：程序的转移概率不会很低；数据分布的离散性较大



7.6.3 避免存储体冲突

第 41 页

1. 体冲突：两个请求要访问同一个体。
2. 减少体冲突次数的一种方法：采用许多体，例如NEC SX/3最多可使用128个体，这种方法存在问题：

假如我们有128个存储体，按字交叉方式工作，并执行以下程序：

```
int x [ 256 ][ 512 ];  
for ( j = 0; j < 512; j = j+1 )  
    for ( i = 0; i < 256; i = i+1 )  
        x [ i ][ j ] = 2 * x [ i ][ j ];
```

因为512是128的整数倍，同一列中的所有元素都在同一个体内，无论CPU或存储系统多么高级，该程序都会在数据Cache不命中时暂停

体内地址	0号体	1号体	2号体	3号体
0	A0	A1	A2	A3
1	A4	A5	A6	A7
2	A8	A9	A10	A11
3	A12	A13	A14	A15

A[0][0]、A[1][0]、A[2][0].....
在0号存储体

7.6.3 避免存储体冲突

第 42 页

3. 解决存储体冲突的方法

■ 软件方法(编译器)

➤ 循环交换优化——避免对同一个体访问

体内地址	0号体	1号体	2号体	3号体
0	A0	A1	A2	A3
1	A4	A5	A6	A7
2	A8	A9	A10	A11
3	A12	A13	A14	A15

```
int x [ 256 ][ 512 ];  
for ( j = 0; j < 512; j = j+1 )  
    for ( i = 0; i < 256; i = i+1 )  
        x [ i ][ j ] = 2 * x [ i ][ j ];
```



```
int x [ 256 ][ 512 ];  
for ( i = 0; i < 256; i = i+1 )  
    for ( j = 0; j < 512; j = j+1 )  
        x [ i ][ j ] = 2 * x [ i ][ j ];
```

A[0][0] A[0][1] A[0][2].....
在0号存储体 在1号存储体 在2号存储体

7.6.3 避免存储体冲突

第 43 页

3. 解决存储体冲突的方法

- 软件方法(编译器)

- 扩展数组的大小, 使之不是2的幂

```
int x [ 256 ][ 512 ];  
for ( j = 0; j < 512; j = j+1 )  
    for ( i = 0; i < 256; i = i+1 )  
        x [ i ][ j ] = 2 * x [ i ][ j ];
```



```
int x [ 257 ][ 513 ];  
for ( j = 1; j < 513; j = j+1 )  
    for ( i = 1; i < 257; i = i+1 )  
        x [ i ][ j ] = 2 * x [ i ][ j ];
```

A[1][1] A[2][1] A[3][1].....
在2号存储体 在3号存储体 在4号存储体

7.6.3 避免存储体冲突

第 44 页

3. 解决存储体冲突的方法

- 硬件方法—使存储体个数为素数（质数）

体内地址 0号体 1号体 2号体 3号体

0	A0	A1	A2	A3
1	A4	A5	A6	A7
2	A8	A9	A10	A11
3	A12	A13	A14	A15



体内地址 0号体 1号体 2号体 3号体 4号体

0	A0	A1	A2	A3	A4
1	A5	A6	A7	A8	A9
2	A10	A11	A12	A13	A14
3	A15	A16	A17	A18	A19

7.6.3 避免存储体冲突

第 45 页

3. 解决存储体冲突的方法

- 体内地址 = 地址A mod (存储体中的字数)可以直接截取

顺序交叉和取模交叉的地址映射举例

体号 $j = A \bmod m$

体内地址 $i = \left\lfloor \frac{A}{m} \right\rfloor$



体内地址	存储体					
	顺序交叉			取模交叉		
	0	1	2	0	1	2
0	0	1	2	0	16	8
1	3	4	5	9	1	17
2	6	7	8	18	10	2
3	9	10	11	3	19	11
4	12	13	14	12	4	20
5	15	16	17	21	13	5
6	18	19	20	6	22	14
7	21	22	23	15	7	23

体号 $j = A \bmod m$

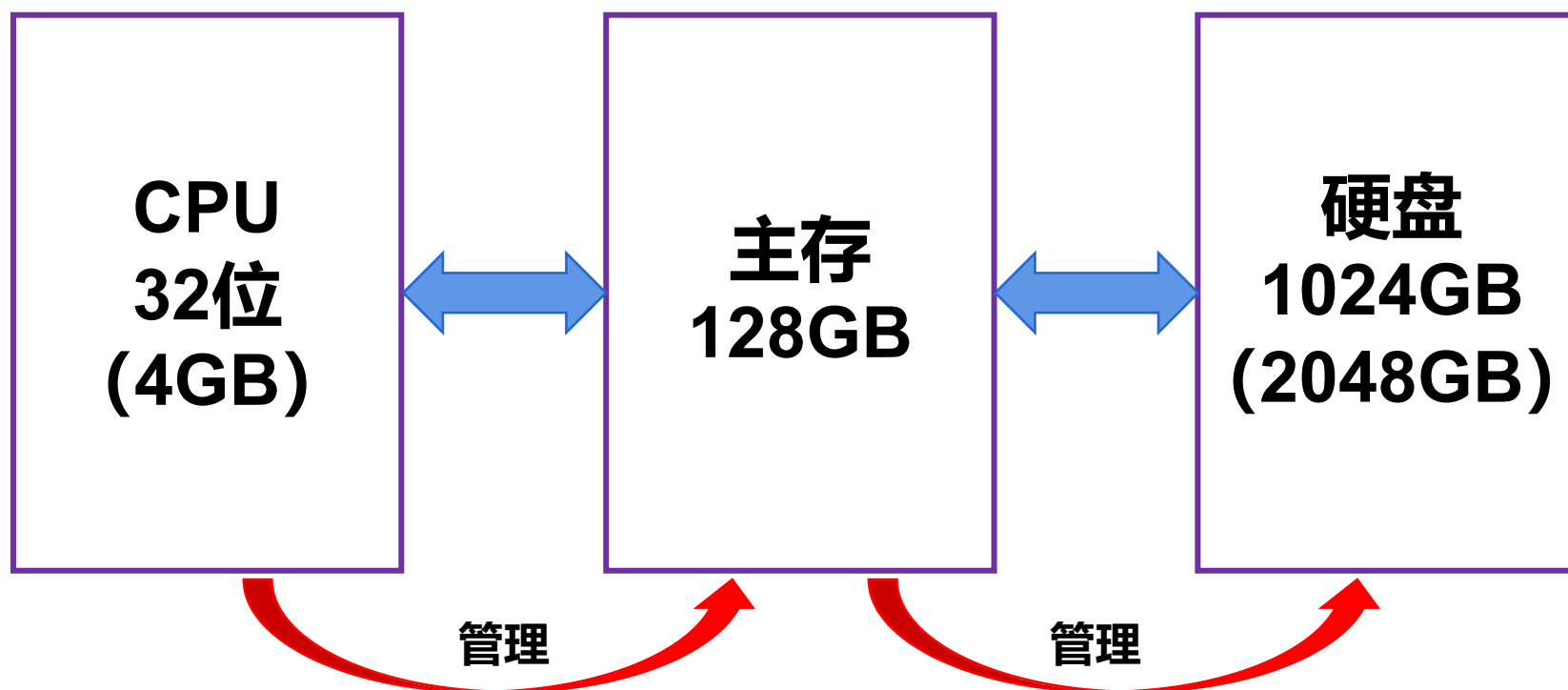
体内地址 $i = A \bmod n$



7.7 虚拟存储器

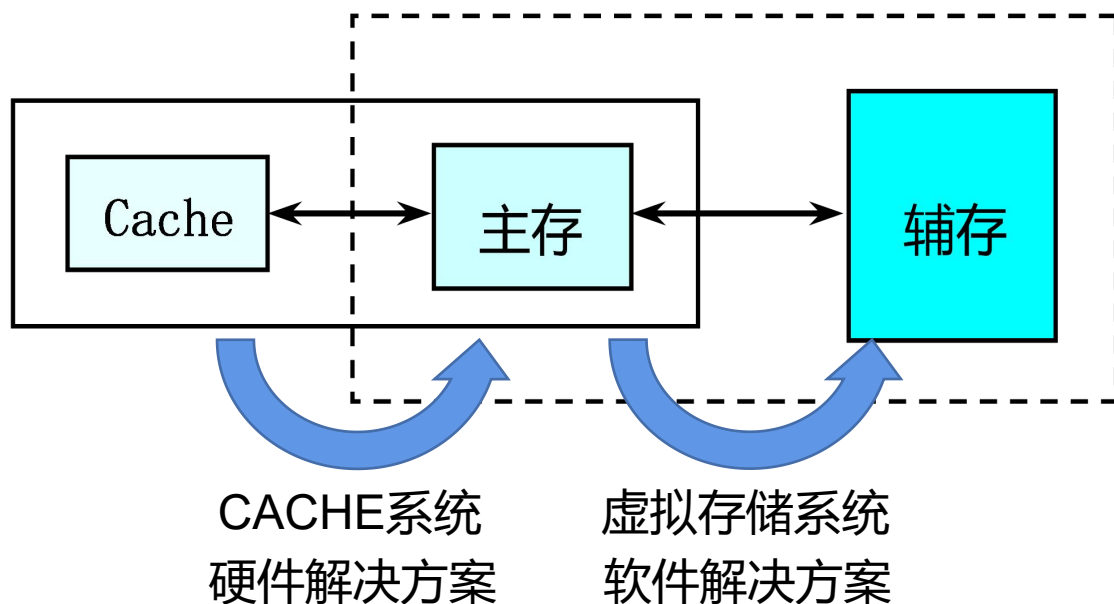
7.7.1 虚拟存储器的基本原理

第 47 页



7.7.1 虚拟存储器的基本原理

第 48 页



虚拟存储器实现了内存扩充功能。但该功能并非是从物理上实际地扩大内存的容量，而是从逻辑上实现对内存容量的扩充。

**核心问题：主存和辅存中的
数据块大小和管理方法**

■ 存在现象：

- 有的作业很大，其所要求的内存空间超过了内存总容量；
- 有大量作业要求运行，但由于内存容量不足以容纳所有这些作业。

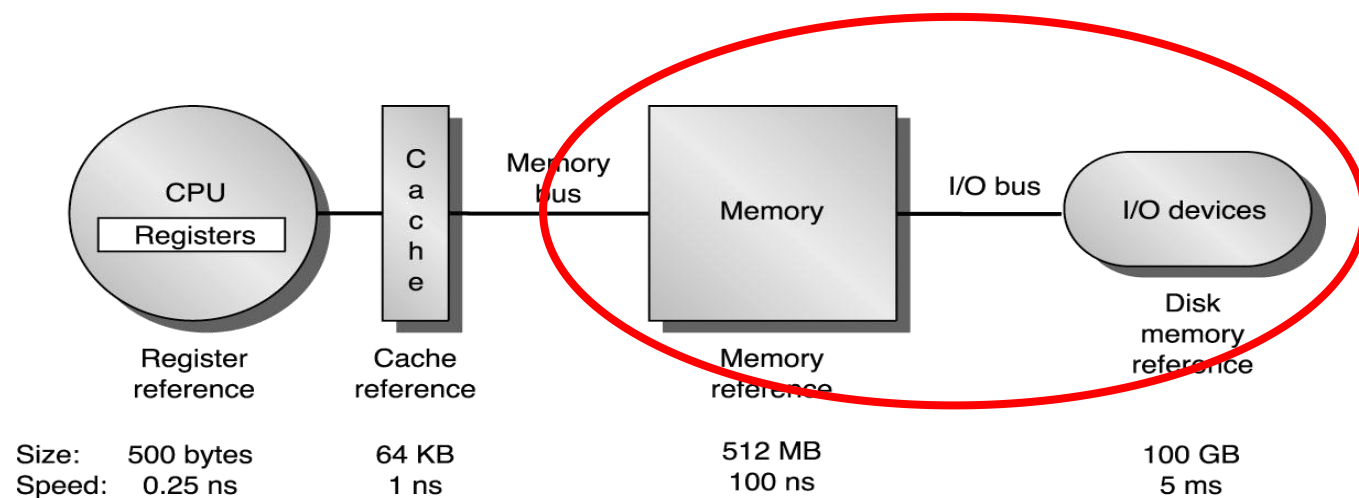
■ 原因：由于内存容量不够大导致的

■ 解决方法：从逻辑上扩充内存容量

7.7.1 虚拟存储器的基本原理

第 49 页

1. 虚拟存储器：在**操作系统及辅助硬件**的管理下，由主存和大容量外存所构成的一个单一的，可直接访问的超大容量的主存储器。
2. 虚拟存储系统也称虚拟存储器、虚拟存储体系，1961年英国曼彻斯特大学 Kilbrn 等人提出，上个世纪70年代广泛地应用于大中型计算机系统中，RISC 处理器开始使用虚拟存储器。



7.7.1 虚拟存储器的基本原理

3. Cache和虚拟存储器的参数取值范围

参数	第一级Cache	虚拟存储器
块（页）大小	16-128字节	4096-65,536字节
命中时间	1-3个时钟周期	100-200个时钟周期
不命中开销	8-200个时钟周期	1,000,000-10,000,000个时钟周期
（访问时间）	(6-160个时钟周期)	(800,000-8,000,000个时钟周期)
（传输时间）	(2-40个时钟周期)	(200,000-2,000,000个时钟周期)
不命中率	0.1-10%	0.00001-0.001%
地址映象	25-45位物理地址到14-20位Cache地址	32-64位虚拟地址到25-45位物理地址

7.7.2 地址映象和变换

第 51 页

虚拟存储器工作原理：工作时，操作系统管理的地址转换硬件检测程序欲访问的虚存地址所在的程序或数据页（或段）是否在主存中，若已在主存中（即命中），则将虚拟地址转换为主存地址，CPU根据主存地址从主存（或Cache-主存）读取程序或读/写数据；若未在主存中（即失效），则将虚存地址转换为外存地址，由操作系统控制把当前要执行的程序或使用的数据从外存调入到快速主存中，大量暂时不用的部分仍放在慢速廉价的外存中。

- **地址空间**：虚拟地址空间、主存地址空间
- **地址类型**：虚拟地址、主存地址
- **地址映象**：把虚拟地址空间映象到主存地址空间
- **地址变换**：在程序运行时，把虚地址变换成主存地址

7.7.2 地址映象和变换

第 52 页

虚地址是用户程序中的逻辑地址，它包括页号和页内地址（页内位移）。区分页号和页内地址的依据是页的大小，页内地址占虚地址的低位部分，页号占虚地址的高位部分。

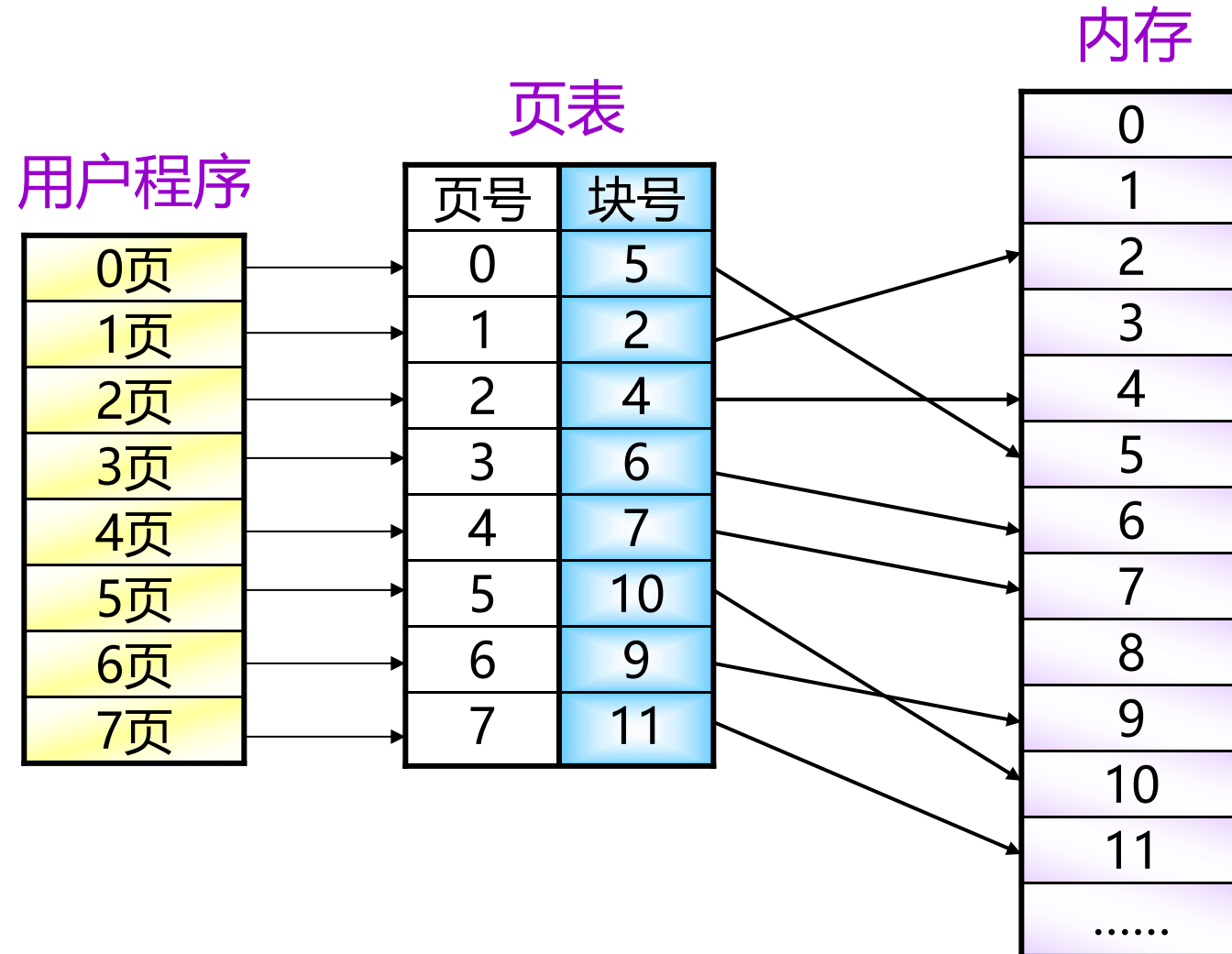
假定页面大小1024字节，虚地址共占用2个字节(16位)

页号 页内地址（位移量）



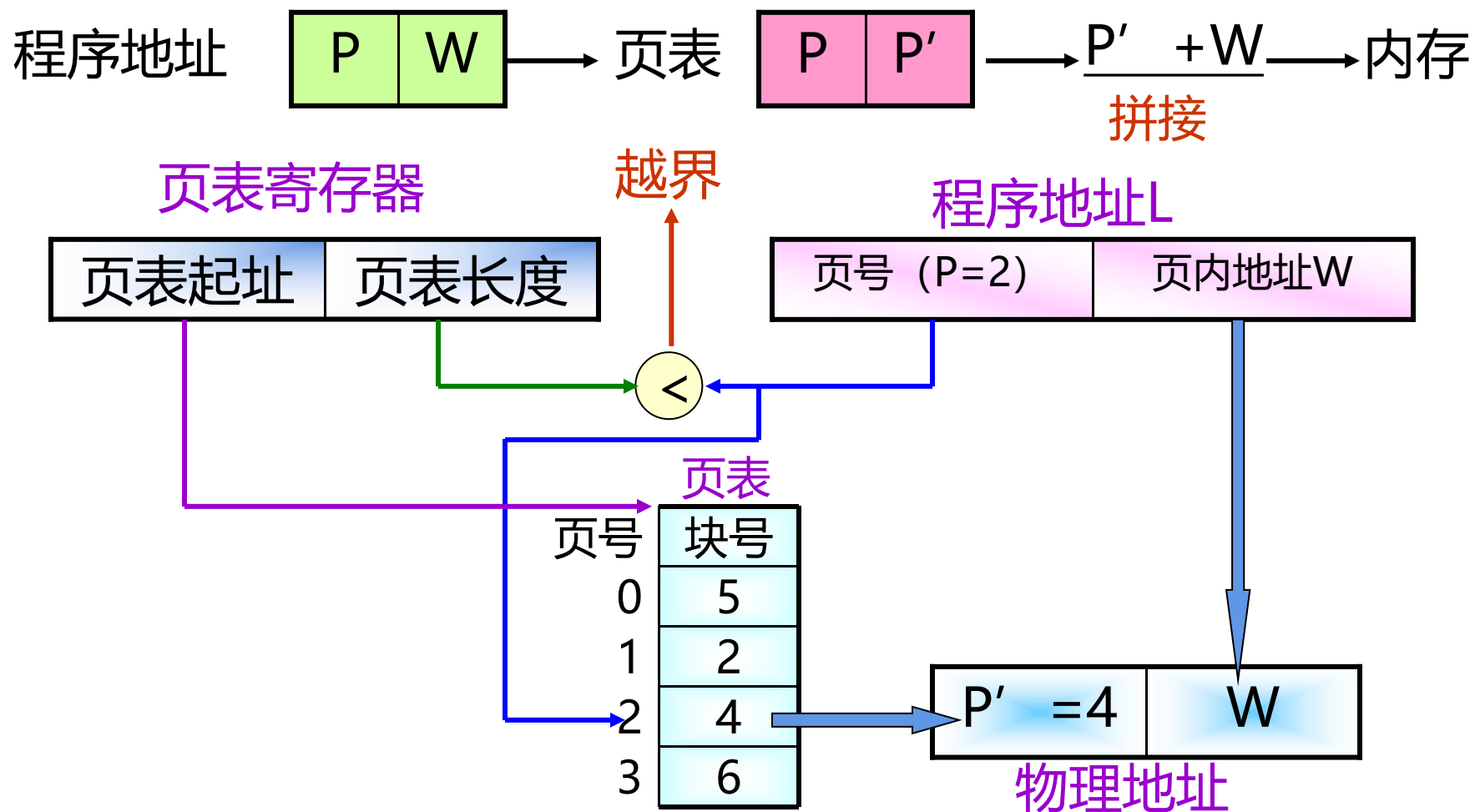
7.7.2 地址映象和变换

第 53 页



7.7.2 地址映象和变换

第 54 页



7.7.2 地址映象和变换

第 55 页

例7.13 有一系统采用页式存储管理，有一作业大小是8KB，页大小为2KB，依次装入内存的第7、9、A、5块，试将虚地址0AFEH，1ADDH转换成内存地址

虚地址0AFEH

0000 1010 1111 1110

P = 1 W = 010 1111 1110

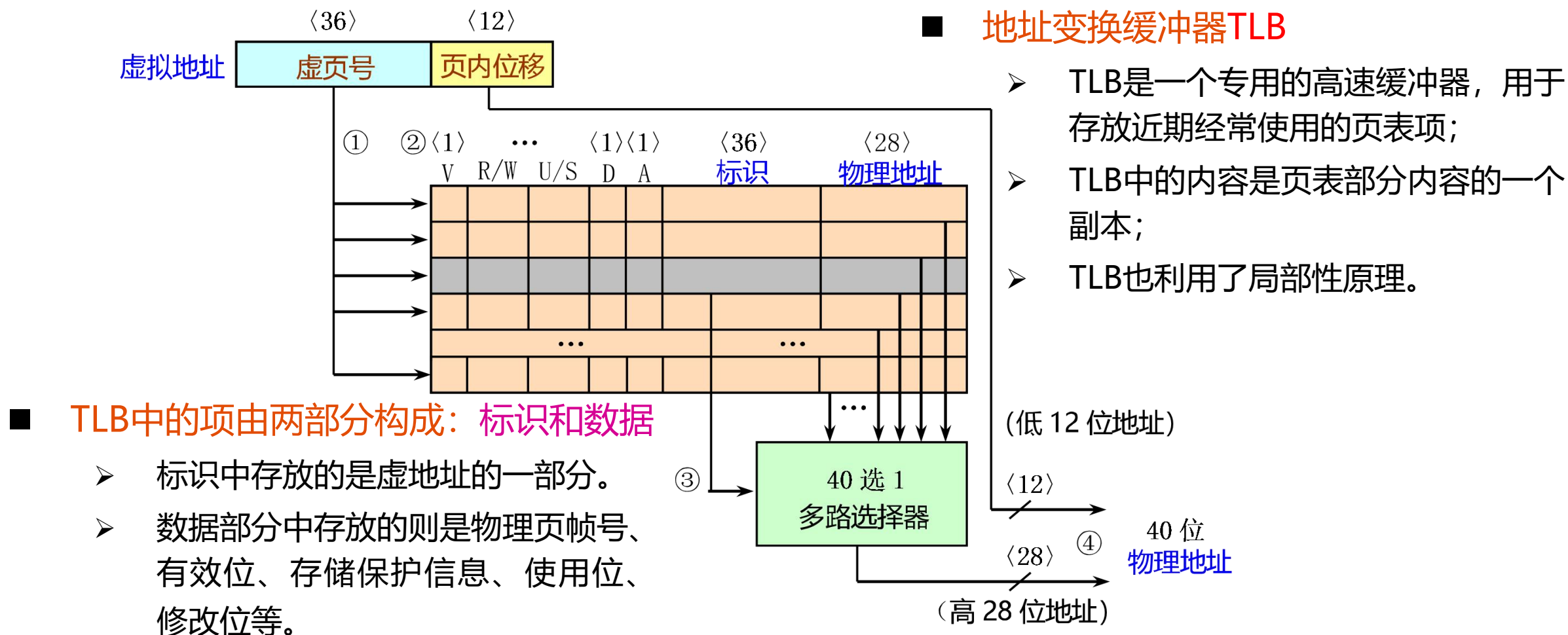
MR = 0100 1010 1111 1110
= 4AFEH

页号	块号
0	7
1	9
2	A
3	5

7.7.2 地址映象和变换

第 56 页

一般TLB比Cache的标识存储器更小、更快。保证TLB的读出操作不会使Cache的命中时间延长。



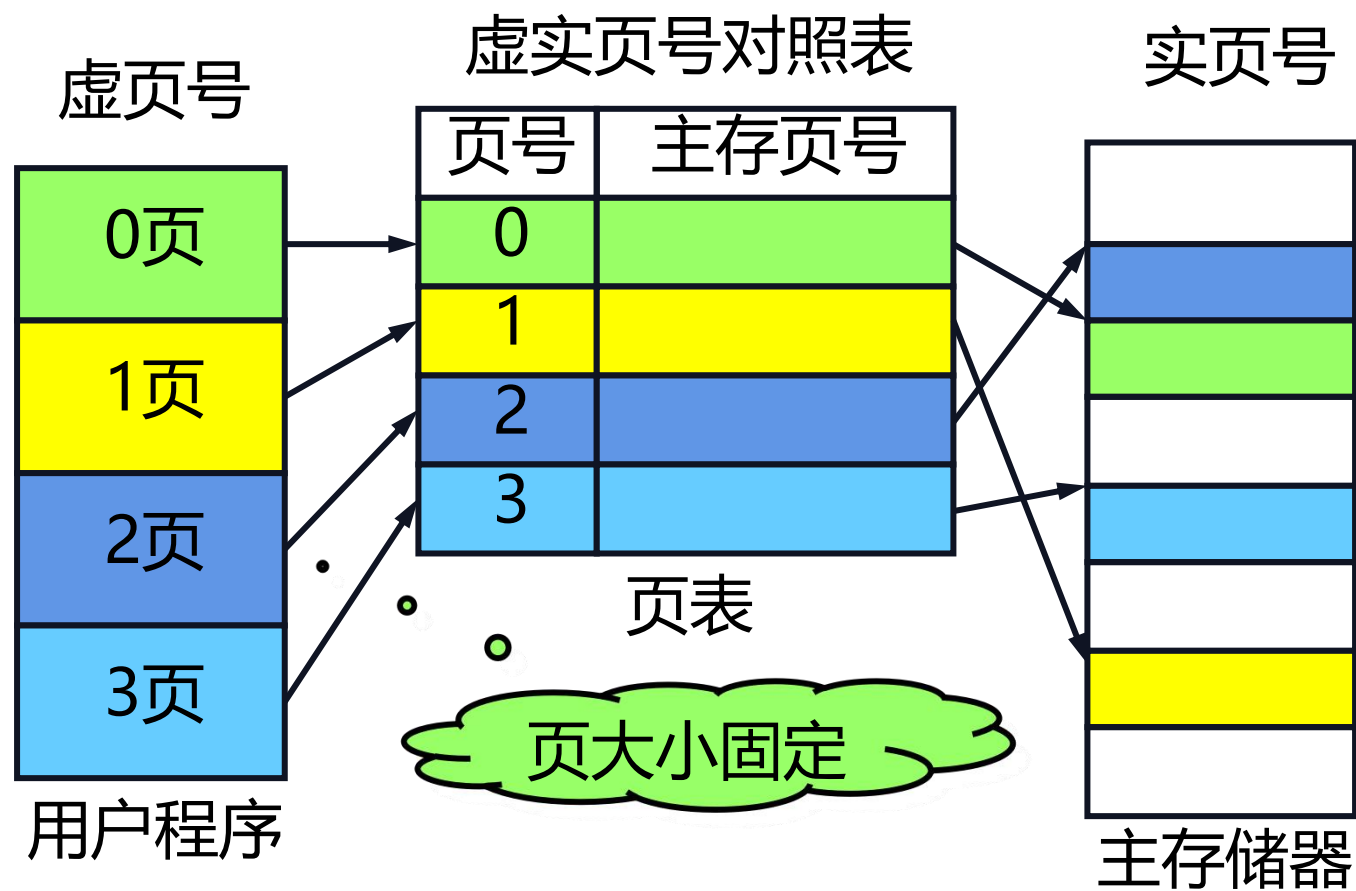
根据所采用的地址映象和变换方法（内部地址变换）不同，有三种不同类型的虚拟存储器：

- [页式虚拟存储器](#)
- [段式虚拟存储器](#)
- [段页式虚拟存储器](#)

7.7.2 地址映象和变换

第 58 页

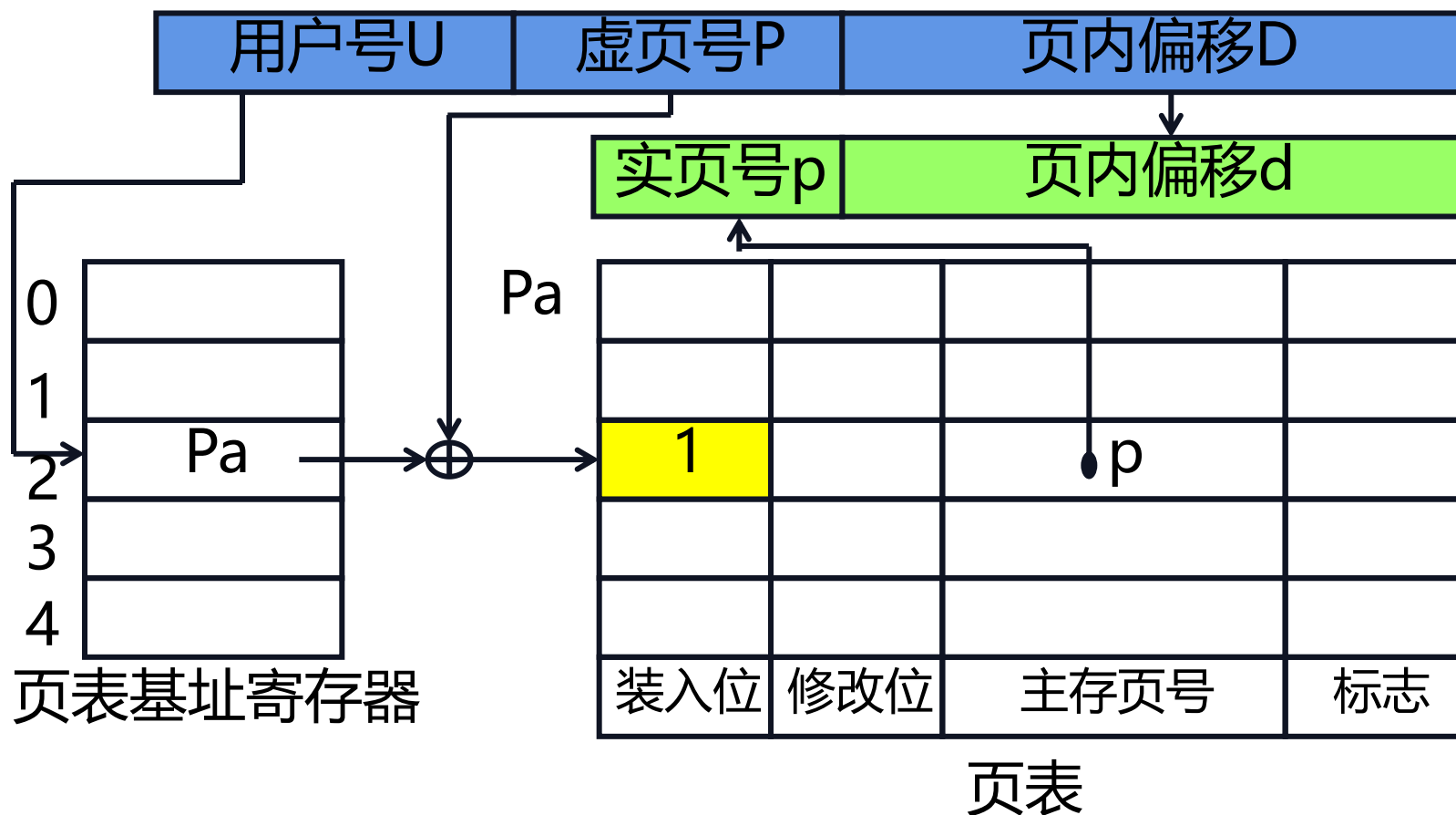
1. 页式虚拟存储系统



7.7.2 地址映象和变换

第 59 页

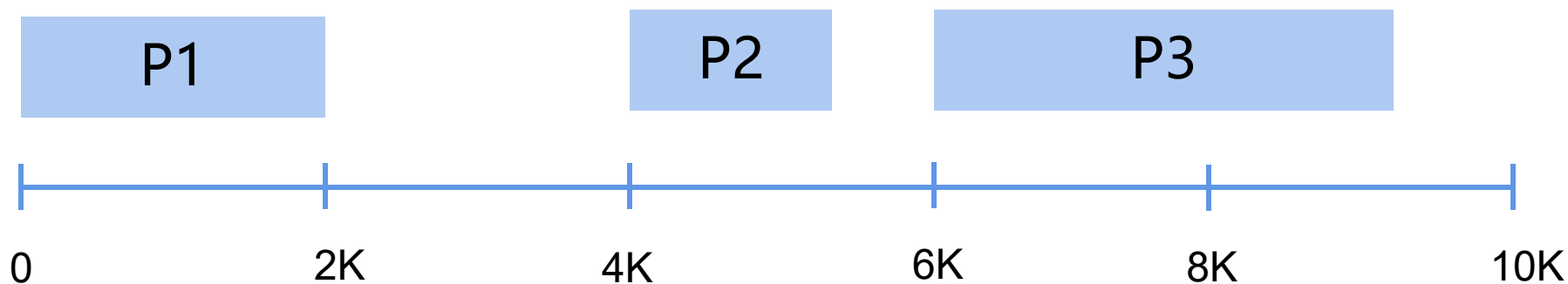
1. 页式虚拟存储系统



7.7.2 地址映象和变换

第 60 页

1. 页式虚拟存储系统



• 优点

- ◆主存储器的利用率比较高;
- ◆页表相对比较简单;
- ◆地址变换的速度比较快;
- ◆对磁盘的管理比较容易。

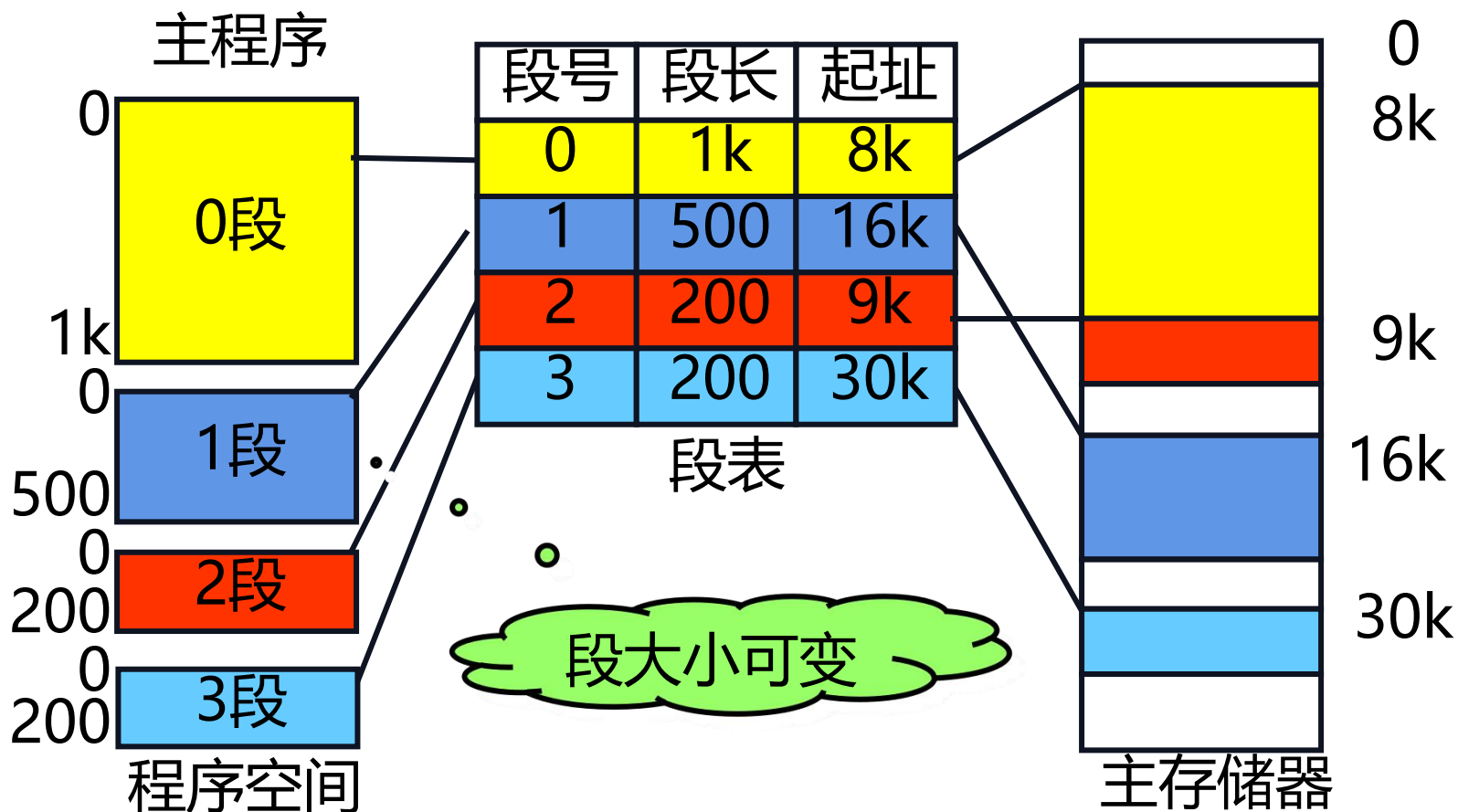
• 缺点

- ◆程序的模块化性能不好;
- ◆页表很长, 需要占用很大的存储空间。

7.7.2 地址映象和变换

第 61 页

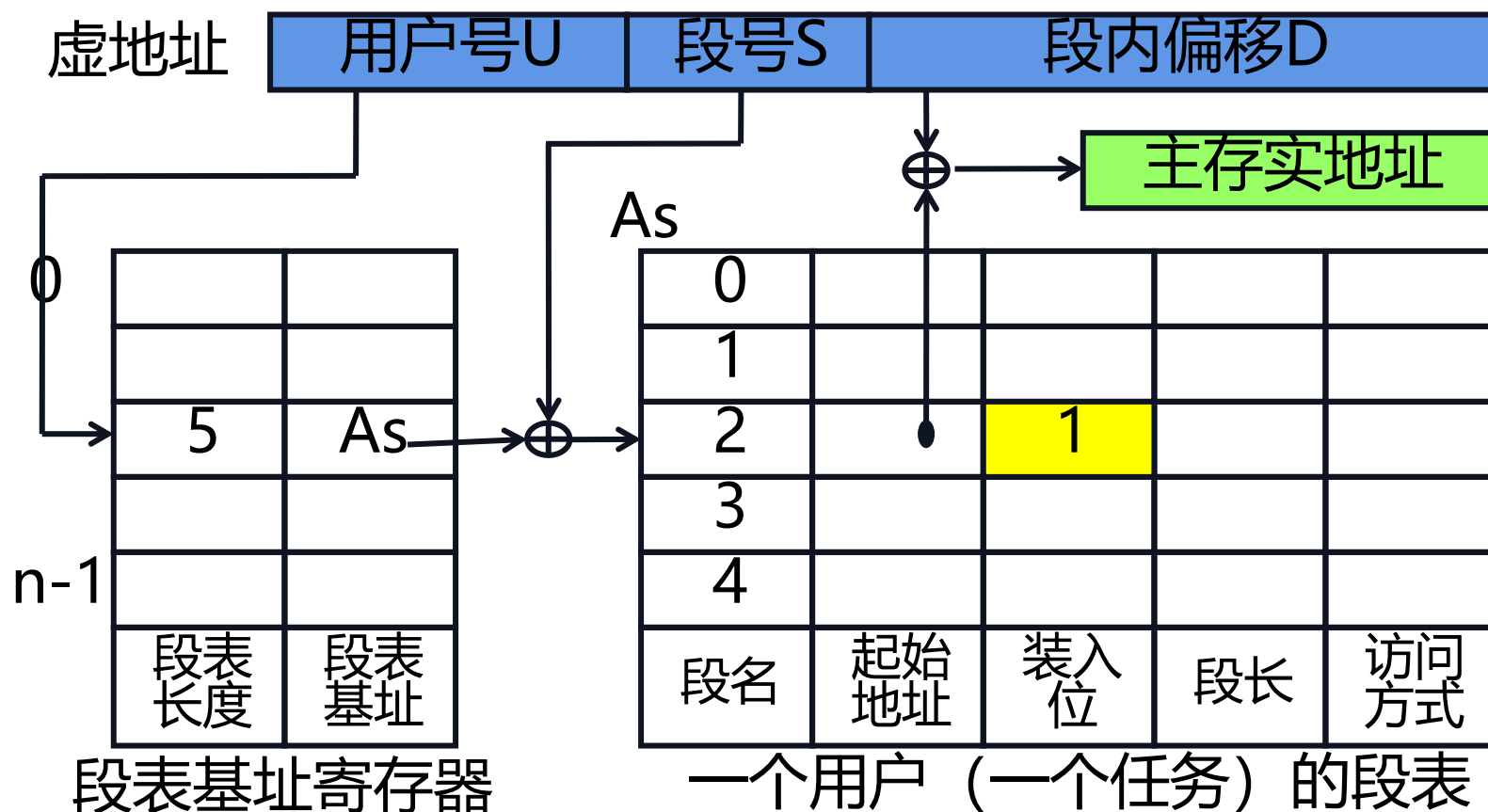
2. 段式虚拟存储系统



7.7.2 地址映象和变换

第 62 页

2. 段式虚拟存储系统



2. 段式虚拟存储系统

• 优点

- 程序的模块化性能好
- 便于程序和数据共享
- 程序的动态链接和调度比较容易
- 便于实现信息保护

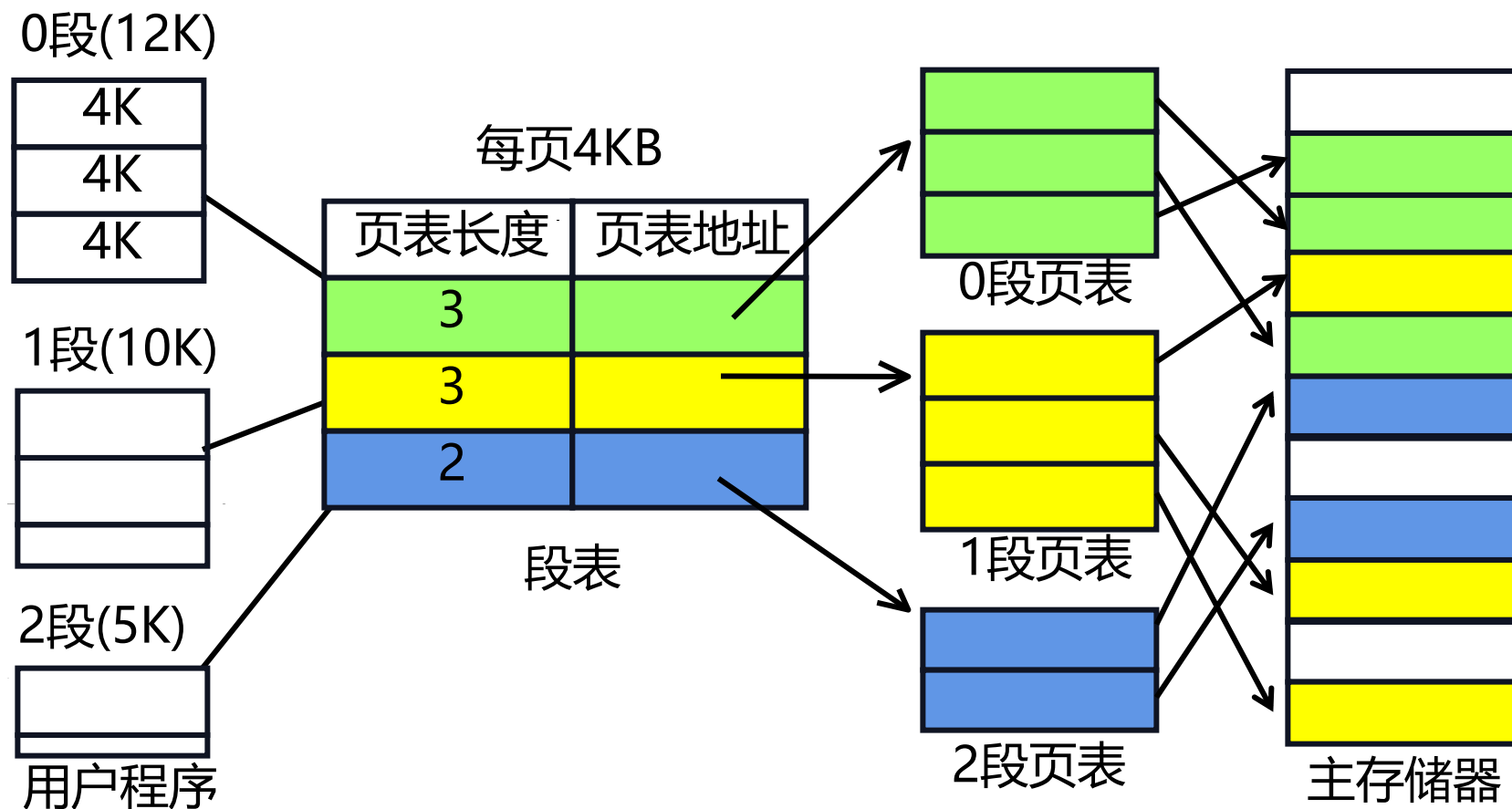
• 缺点

- 地址变换所花费的时间比较长，做两次加法运算
- 主存储器的利用率往往比较低
- 存储管理比较困难

7.7.2 地址映象和变换

第 64 页

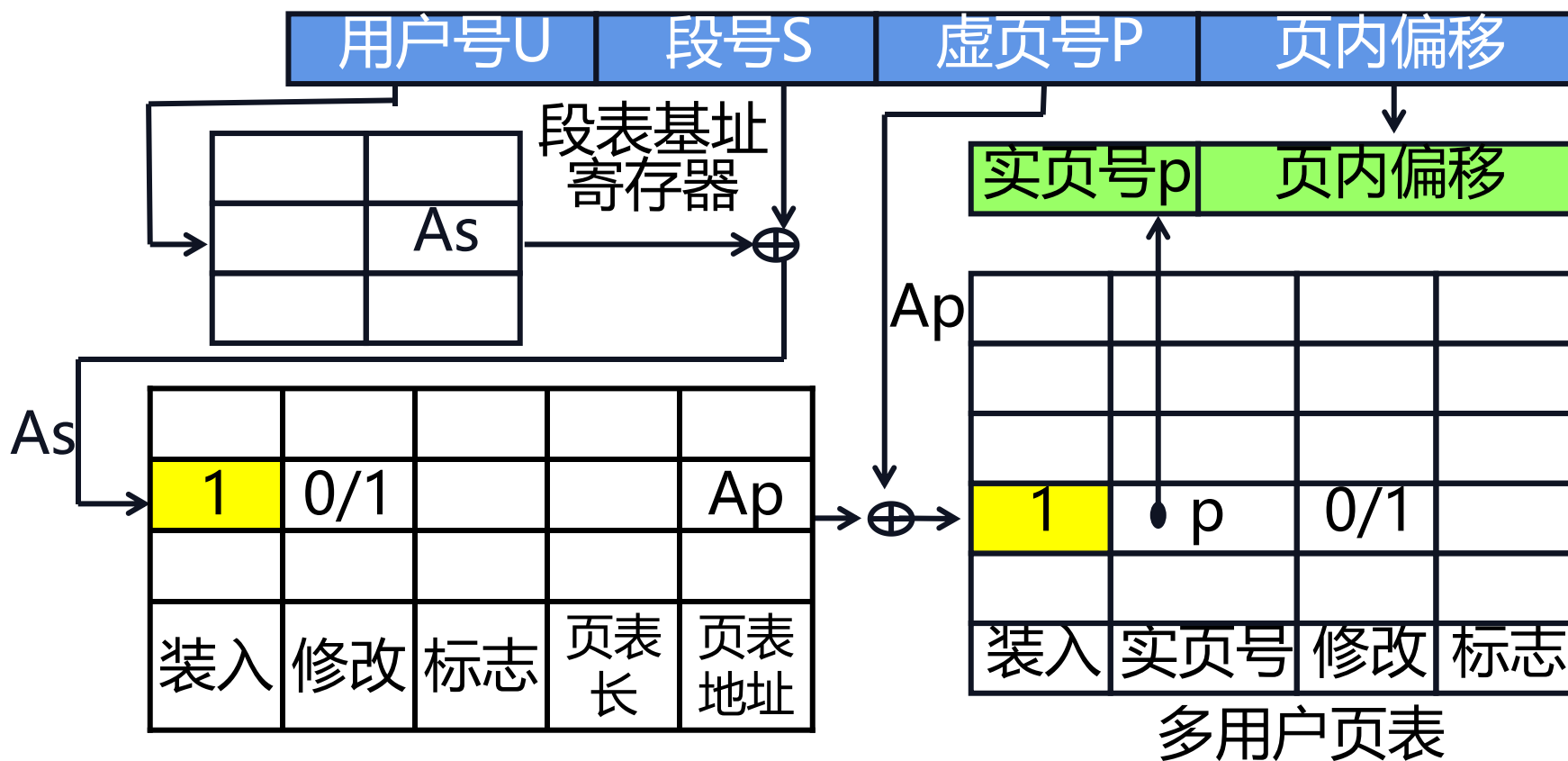
3. 段页式虚拟存储系统



7.7.2 地址映象和变换

第 65 页

3. 段页式虚拟存储系统



■ 为什么需要页面替换？

当发生页面失效时，要从磁盘中调入一页到主存。如果主存所有页面都已经被占用，此时必须从主存储器中淘汰掉一个不常使用的页面，以便腾出主存空间来存放新调入的页面。

■ 评价页面替换算法好坏的标准

一是命中率要高，二是算法要容易实现。

7.7.3 页面替换算法

第 67 页

- 随机算法 (RAND 算法)
- 先进先出算法 (FIFO算法)
- 近期最少使用算法 (LFU算法)
- 最久没有使用算法 (LRU算法)
- 最优替换算法 (OPT算法)

7.7.3 页面替换算法

第 68 页

■ 随机算法

- 利用软件或硬件的随机数发生器来确定主存中要被替换的页面。
- 特点：算法简单，容易实现；但没有利用历史信息，没有反映程序的局部性，命中率低。

■ 先进先出算法

- 选择最先调入主存的页面作为要被替换的页面。
- 特点：比较容易实现，利用了历史信息，但没有反映程序的局部性。

■ 近期最少使用算法

- 选择近期最少访问的页面作为要被替换的页面。
- 特点：既充分利用了历史信息，又反映了程序的局部性，但实现起来非常困难。

7.7.3 页面替换算法

第 69 页

■ 最久没有使用算法

- 选择近期最久没有被访问过的页面作为要被替换的页面。
- 特点：既充分利用了历史信息，又反映了程序的局部性，而且实现起来比较容易。

■ 最优替换算法

- 选择将来最久不被访问的页面作为要被替换的页面。
- 特点：OPT算法是一种理想化的算法，可用来作为评价其它页面替换算法好坏的标准。

7.7.3 页面替换算法

第 70 页

一个程序共有5个页面组成，程序执行过程中的页地址流如下：

P1, P2, P1, P5, P4, P1, P3, P4, P2, P4

假设分配给这个程序的主存储器共有3个页面。给出FIFO、LRU、OPT 三种页面替换算法对这3页主存的使用情况，包括调入、替换和命中等。

7.7.3 页面替换算法

时间	1	2	3	4	5	6	7	8	9	10	实际命中次数
页地址流	P1	P2	P1	P5	P4	P1	P3	P4	P2	P4	
FIFO 算法	1	1	1	1 *	4	4	4 *	4 *	2	2	2次
		2	2	2	2 *	1	1	1	1 *	4	
				5	5	5 *	3	3	3	3*	
	调入	调入	命中	调入	替换	替换	替换	命中	替换	替换	
LRU 算法	1	1	1	1	1 *	1	1	1 *	2	2	4次
		2	2	2 *	4	4	4 *	4	4	4	
				5	5	5 *	3	3	3 *	3 *	
	调入	调入	命中	调入	替换	命中	替换	命中	替换	命中	
OPT 算法	1	1	1	1	1	1 *	3 *	3 *	3 *	3 *	5次
		2	2	2	2 *	2	2	2	2	2	
				5 *	4	4	4	4	4	4	
	调入	调入	命中	调入	替换	命中	替换	命中	命中	命中	

7.7.4主存命中率提升方法

第 72 页

影响主存命中率的主要因素：

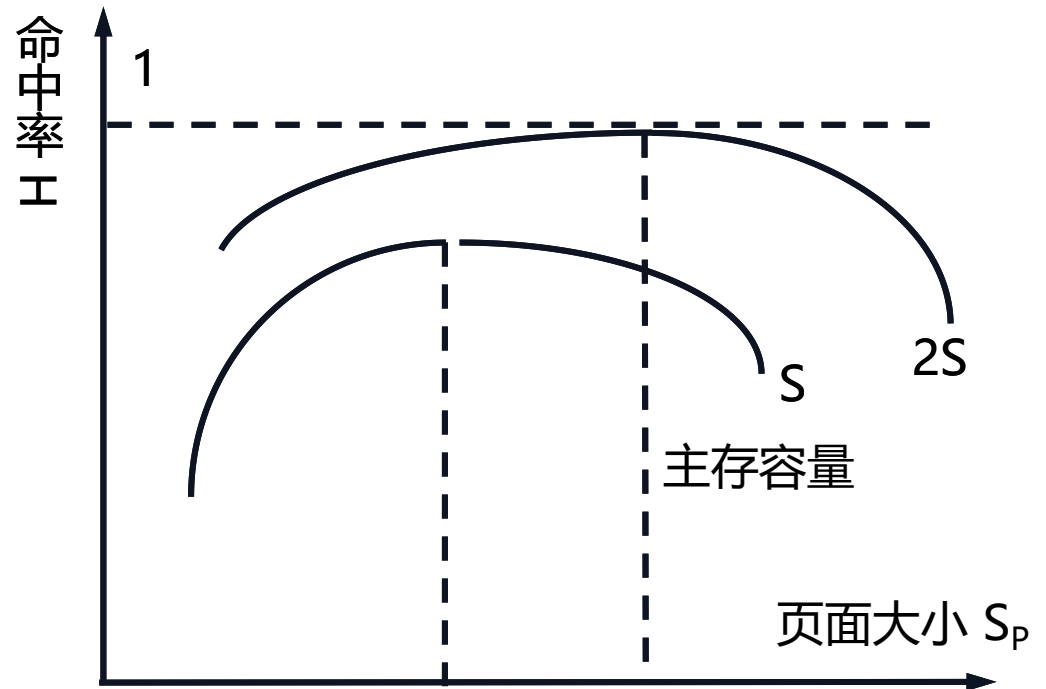
- 页面大小
- 主存容量
- 所采用的页面调度算法

7.7.4主存命中率提升方法

第 73 页

1. 页面大小

页面大小的选择一般要通过通过对典型程序的模拟实验来确定，早期一般为1KB，目前大多为4KB、8KB、16KB。

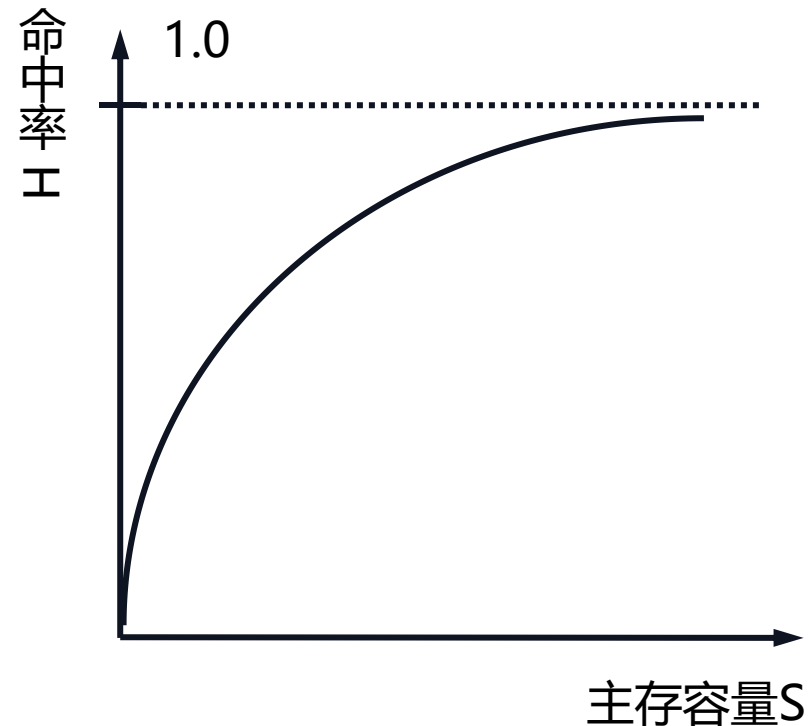


7.7.4主存命中率提升方法

第 74 页

2. 主存容量

主存命中率 H 随着分配给该程序的主存容量 S 的增加而单调上升。



选择题

第 75 页

1、虚拟存储器主要是为了 (A)

- A. 扩大存储系统的容量
- B. 提高存储系统的速度
- C. 扩大存储系统的容量和提高存储系统的速度
- D. 便于程序的访存操作

2、与全相联映像相比，组相联映像的优点是 (A)

- A. 目录小
- B. 块冲突概率低
- C. 命中率高
- D. 主存利用率高

3、按Cache地址映像的块冲突概率从高到低的顺序 (C)

- A. 全相联映像、直接映像、组相联映像
- B. 组相联映像、直接映像、全相联映像
- C. 直接映像、组相联映像、全相联映像
- D. 全相联映像、组相联映像、直接映像

4、对于采用组相联映像、LRU替换算法的Cache存储器来说，不影响Cache命中率的是 (C)

- A. 增加Cache中的块数
- B. 增大组的大小
- C. 增大主存容量
- D. 增大块的大小

1、写出三级Cache的平均访问时间的公式？

解：核心关键 (1) 平均访存时间=命中时间+不命中率×不命中开销

(2) 只有第I层的不命中时才会访问第I+1层

设三级Cache的访问时间分别为 T_{A1} 、 T_{A2} 、 T_{A3} ；命中率分别为 H_1 、 H_2 、 H_3 ；第三级Cache的不命中开销为 P 。

$$\text{平均访问时间 } T_A = T_{A1} \times H_1 + (1 - H_1) \times \{ T_{A2} \times H_2 + (1 - H_2) \times [T_{A3} \times H_3 + (1 - H_3) \times P] \}$$

2、VAX-11/780在Cache命中时的指令平均执行时间是9.5时钟周期，Cache不命中时间是5个时钟周期。假设不命中率是12%，每条指令平均访存2.5次。试计算考虑了Cache不命中时的指令平均执行时间。它比Cache命中时的指令平均执行时间延长了百分之几？

解： Cache不命中时， $2.5 \times 12\% \times 5 = 1.5$

(1) 考虑了Cache不命中时的指令平均执行时间： $9.5 + 1.5 = 11$ （个时间周期）

(2) $(11 - 9.5) \div 9.5 = 15.79\%$

它比Cache命中时的指令平均执行时间延长了15.79%。

3、我们考虑某一个机器，假设Cache读不命中开销为25个时钟周期，写不命中开销为70个时钟周期，当不考虑存储器停顿时，所有指令的执行时间都是2.0个时钟周期，Cache的读不命中率和写不命中率均为4%，平均每条指令读存储器0.8次，写存储器0.5次，试分析考虑Cache的不命中后，Cache对性能的影响。

解：（1）平均每条指令存储器停顿时钟周期数=读次数×读不命中率×读不命中开销
+写次数×写不命中率×写不命中开销
=0.8×4%×25+0.5×4%×70=2.2

（2）CPU时间=IC×[CPI+（存储停顿周期数/指令数）]×时钟周期时间

（3）考虑Cache的不命中后，性能为：

CPU时间=IC×（2.0+2.2）×时钟周期时间=IC×4.2×时钟周期时间

当考虑了Cache的不命中影响后，CPI从理想计算机的2.0增加到4.2，是原来的2.1倍。

4、假设对指令Cache的访问占全部访问的75%；而对数据Cache的访问占全部访问的25%。Cache的命中时间为1个时钟周期，不命中开销为50个时钟周期，在统一Cache中一次load或store操作访问Cache的命中时间都要增加一个时钟周期，32KB的指令Cache的不命中率为0.15%，32KB的数据Cache的不命中率为3.77%，64KB的统一Cache的不命中率为0.95%。又假设采用写直达策略，且有一个写缓冲器，并且忽略写缓冲器引起的等待。试问指令Cache和数据Cache均为32KB的分离Cache和容量为64KB的统一Cache相比，哪种Cache的不命中率更代？两种情况下平均访存时间各是多少？

解：（1）根据题意，75%的访存为取指令，则分离Cache的总体不命中率为：

$$(75\% \times 0.15\%) + (25\% + 3.77\%) = 1.055\%$$

容量64KB的统一Cache的不命中率为0.95%，略低一些。

（2）平均访存时间=指令访存所占的百分比×（读命中时间+读不命中率×不命中开销）+
数据访存所占的百分比×（写命中时间+写不命中率×不命中开销）

4、假设对指令Cache的访问占全部访问的75%；而对数据Cache的访问占全部访问的25%。Cache的命中时间为1个时钟周期，不命中开销为50个时钟周期，在统一Cache中一次load或store操作访问Cache的命中时间都要增加一个时钟周期，32KB的指令Cache的不命中率为0.15%，32KB的数据Cache的不命中率为3.77%，64KB的统一Cache的不命中率为0.95%。又假设采用写直达策略，且有一个写缓冲器，并且忽略写缓冲器引起的等待。试问指令Cache和数据Cache均为32KB的分离Cache和容量为64KB的统一Cache相比，哪种Cache的不命中率更代？两种情况下平均访存时间各是多少？

解：分离Cache的平均访存时间= $75\% \times (1 + 0.15\% \times 50) + 25\% \times (1 + 3.77\% \times 50) = 1.5275$

统一Cache的平均访存时间= $75\% \times (1 + 0.95\% \times 50) + 25\% \times (1 + 1 + 0.95\% \times 50) = 1.725$

尽管分离Cache的实际不命中率比统一Cache的高，但其平均访存时间反而较低，且分离Cache提供了两个端口，消除了结构相关。

5、假设存储系统在延迟30个时钟周期后，每两个周期能送出16个字节，即经过32个时钟周期，它可提供16个字节；经过34个时钟周期，可提供32个字节；以此类推。命中时间与块大小无关，为1个时钟周期，分别计算下列各种容量的Cache的平均访存时间。①块大小为32个字节，Cache容量为1KB，不命中率13.34%；②块大小为32个字节，Cache容量为4KB，不命中率为7.24%；③块大小为64个字节，Cache容量为16KB，不命中率为2.64%；④块大小为128个字节，Cache容量为16KB，不命中率为2.77%

解：①不命中开销 $=30+32\div16\times2=34$

平均访存时间 $=1+(13.34\%\times34)=5.54$ 个时钟周期

②平均访存时间 $=1+(7.24\%\times34)=3.462$ 个时钟周期

③不命中开销 $=30+64\div16\times2=38$

平均访存时间 $=1+(2.64\%\times38)=2.003$ 个时钟周期

④不命中开销 $=30+128\div16\times2=46$

平均访存时间 $=1+(2.77\%\times46)=2.274$ 个时钟周期

6、Alpha AXP 21064中，16KB指令Cache的不命中率为0.64%，命中时间为1个时钟周期，不命中开销为60个时钟周期。假设采用指令预取技术后，预取命中率为30%。当指令不在指令Cache里，而在预取缓冲器中找到时，需要多花一个时钟周期，其实际不命中率是多少？

解：

平均访存时间=命中时间+不命中率×[预取命中率×预取开销+(1-预取命中率)×不命中开销]

平均访存时间=1+0.64%×[30%×1+70%×60]=1.277

平均访存时间=命中时间+不命中率×不命中开销

则：不命中率=（平均访存时间-命中时间）/不命中开销=（1.277-1）/60=0.46%

应用题

第 83 页

7、假设在3000次访存中，第一级Cache不命中110次，第二级Cache不命中55次，问：在这种情况下该Cache系统的局部不命中率和全局不命中率各是多少？

解：第一级Cache的全局不命中率和局部不命中率= $110/3000=3.67\%$

第二级Cache的局部不命中率= $55/110=50\%$

第二级Cache的全局不命中率= $55/3000=1.83\%$

8、在三级Cache中，第一级Cache、第二级Cache、第三级Cache的局部不命中率分别为4%、30%和50%。它们的全局不命中率各是多少？

解：（1）第一级全局不命中率=局部不命中率=4%

（2）第二级的全局不命中率=第一级局部不命中率×第二级局部不命中率= $4\% \times 30\% = 1.2\%$

（3）第三级的全局不命中率=第一级局部不命中率×第二级局部不命中率×第三级局部不命中率= $4\% \times 30\% \times 50\% = 0.6\%$

9、给定以下的假设，试计算直接映像Cache和两路组相联Cache的平均访问时间以及CPU性能，由计算结果能得出什么结论？

- (1) 理想Cache情况下的CPI为2.0，时钟周期为2ns，平均每条指令访存1.2次；
- (2) 两路Cache容量均为64KB，块大小都是32B；
- (3) 组相联Cache中的多路选择器使CPU的时钟周期增加了10%；
- (4) 这两种Cache的不命中开销都是80ns；
- (5) 命中时间为1个时钟周期；
- (6) 64KB直接映像Cache的不命中率1.4%，64KB两路组相联Cache的不命中率为1.0%。

解：(1) 平均访问时间 $_{1-路} = 2.0 + 1.4\% \times 80 = 3.12\text{ns}$
平均访问时间 $_{2-路} = 2.0 \times (1 + 10\%) + 1.0\% \times 80 = 3.0\text{ns}$

两路组相联的平均访问时间比较小。

9、给定以下的假设，试计算直接映像Cache和两路组相联Cache的平均访问时间以及CPU性能，由计算结果能得出什么结论？

- (1) 理想Cache情况下的CPI为2.0，时钟周期为2ns，平均每条指令访存1.2次；
- (2) 两路Cache容量均为64KB，块大小都是32B；
- (3) 组相联Cache中的多路选择器使CPU的时钟周期增加了10%；
- (4) 这两种Cache的不命中开销都是80ns；
- (5) 命中时间为1个时钟周期；
- (6) 64KB直接映像Cache的不命中率1.4%，64KB两路组相联Cache的不命中率为1.0%。

解：(2) $\text{CPU}_{\text{时间}} = \text{IC} \times ((\text{CPI}_{\text{执行}} \times \text{时钟周期}) + (\text{每条指令访存次数} \times \text{不命中率} \times \text{不命中时间开销}))$

$$\text{CPU}_{\text{时间1-路}} = \text{IC} \times (2.0 \times 2 + 1.2 \times 0.014 \times 80) = 5.344 \times \text{IC}$$

$$\text{CPU}_{\text{时间2-路}} = \text{IC} \times (2.2 \times 2 + 1.2 \times 0.01 \times 80) = 5.36 \times \text{IC}$$

直接映像Cache
的CPU性能更高。

10、设主存每个分体的存储周期为 $2\mu s$ ，存储字长为 $4B$ ，采用 m 个分体低位交叉编址。由于各种原因，主存实际带宽只能达到最大带宽的 0.6 倍，现要求主存实际带宽为 $4MB/s$ ，问主存分体数应取多少？

解： m 个分体低位交叉编址存储器的最大带宽为：

$$\text{分体数} \times \text{单体带宽} = m \times \frac{\text{存储字长}}{\text{存储周期}} = m \times \frac{4B}{2\mu s}$$

实际带宽为：

$$0.6 \times \text{最大带宽} = 0.6 \times m \times \frac{4B}{2\mu s} \geq 4B/\mu s$$

$$m \geq 3.33 \approx 4$$

1. 存储系统的基本知识：层次结构、性能参数、三级存储系统、关键问题
2. Cache基本知识：基本结构与原理、映像规则、查找方法、替换算法、写策略、性能分析、改进技术
3. 降低Cache不命中率：不命中类型、增加大小与容量、提高相联度、伪CACHE、硬件预取、编译优化、牺牲CACHE
4. 减少Cache不命中开销：两级CACHE、让读不命中优先于写、写缓冲合并、请求字处理、非阻塞CACHE
5. 减少命中时间：虚拟CACHE、访问流水线、踪迹CACHe
6. 并行主存系统：单体多字存储器、多体交叉存储器
7. 虚拟存储器：TLB、替换算法

第8章 输入与输出系统



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

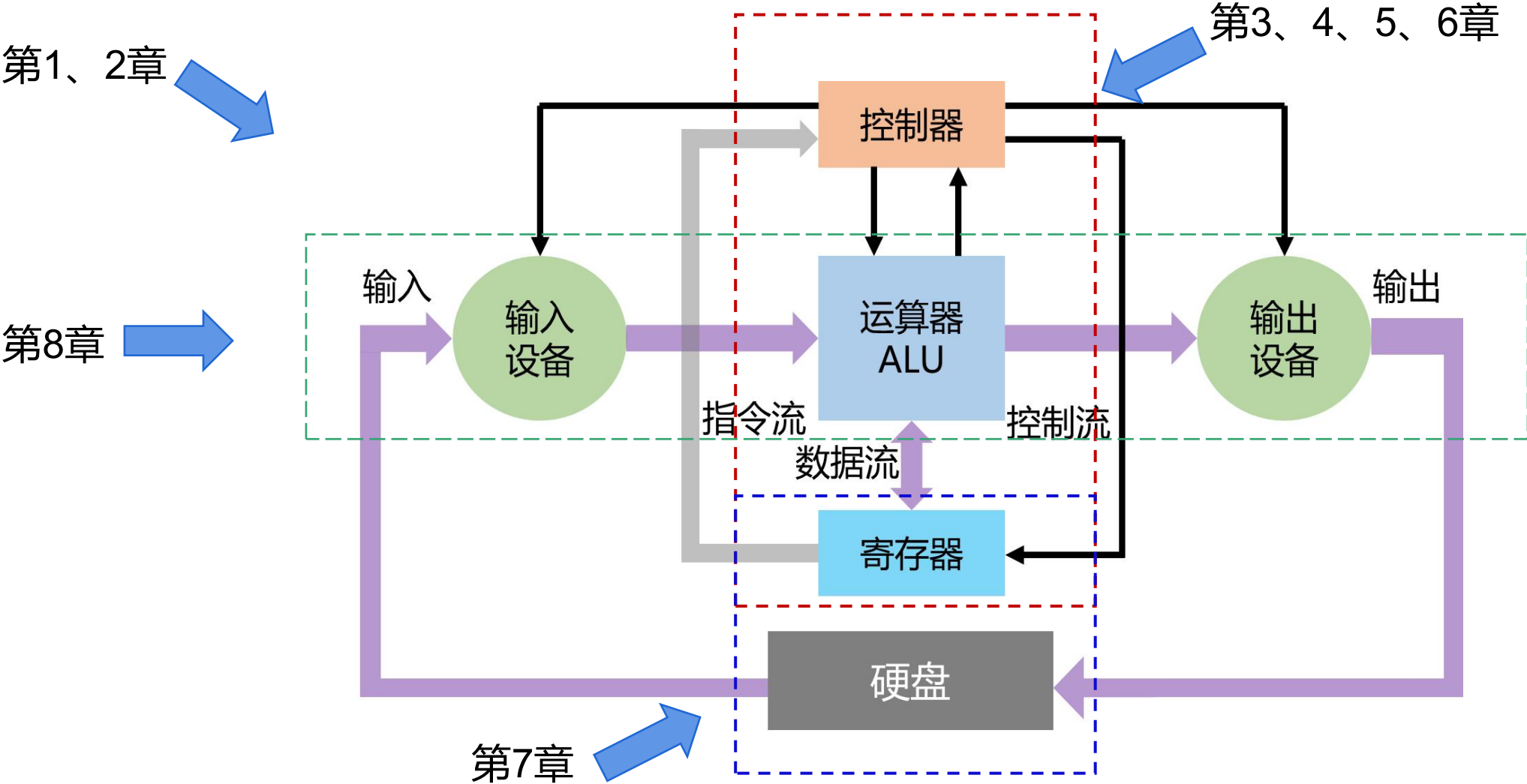
第8章 输入输出系统

2025年春季学期

- 8. 1 [I/O系统的性能](#)
- 8. 2 [总线](#)
- 8. 3 [通道处理机](#)
- 8. 4 [I/O与操作系统](#)

8.1 I/O系统的性能

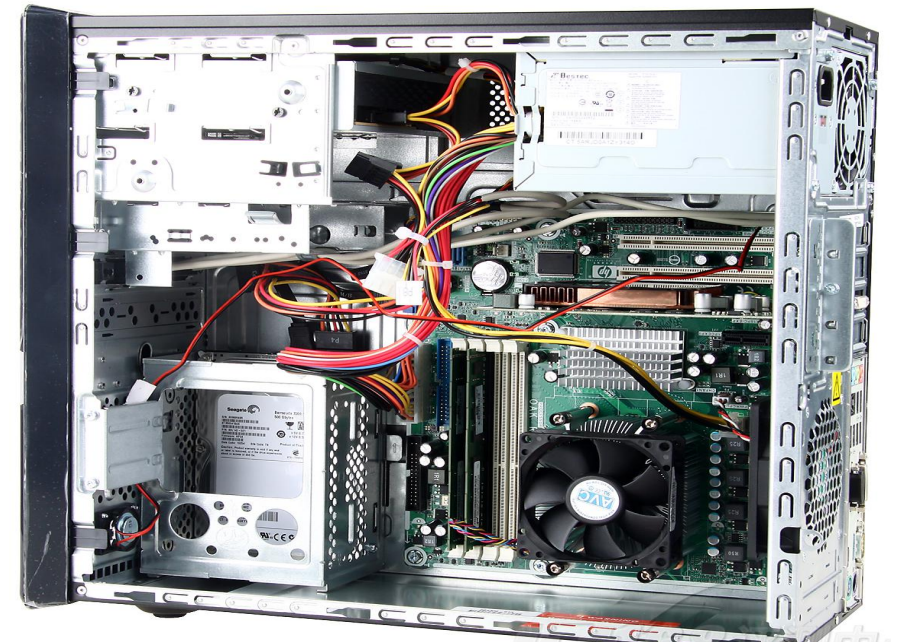
8.1 I/O系统的性能



8.1 I/O系统的性能

第 93 页

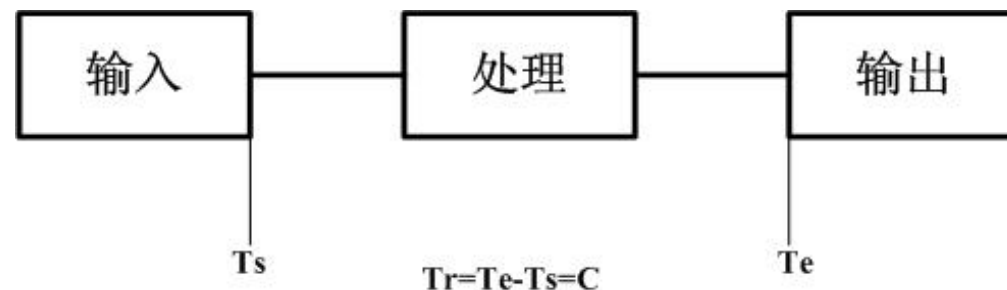
1. 输入/输出系统简称I/O系统，主要包括：
 - I/O设备
 - I/O设备与处理机的连接
2. I/O系统是计算机系统中的一个重要组成部分
 - 完成计算机与外界的信息交换
 - 给计算机提供大容量的外部存储器
3. 按照主要完成的工作进行分类：
 - 存储I/O系统（本章内容）
 - 通信I/O系统



8.1 I/O系统的性能

第 94 页

4. I/O系统的性能对CPU的性能有很大的影响，若两者的性能不匹配，I/O系统就有可能成为整个系统的瓶颈
5. 系统的响应时间(衡量计算机系统的一个更好的指标)
 - 从用户输入命令开始，到得到结果所花费的时间 T_r
 - 由两部分构成： $T_C + T_I$
 - I/O系统的响应时间 T_I
 - CPU的处理时间 T_C
6. 多进程技术只能够提高系统吞吐率，并不能够减少系统响应时间



7. 评价I/O系统性能的参数主要有：

- 连接特性：哪些I/O设备可以和计算机系统相连接
- I/O系统的容量：I/O系统可以容纳的I/O设备数
- 响应时间：从识别一个外部事件到做出响应的的时间
- 吞吐率：单位时间内系统所能完成的任务数量

8. 另一种衡量I/O系统性能的方法：

- 考虑I/O操作对CPU的打扰情况，即考查某个进程在执行时，由于其他进程的I/O操作，使得该进程的执行时间增加了多少。

8. 2 总线

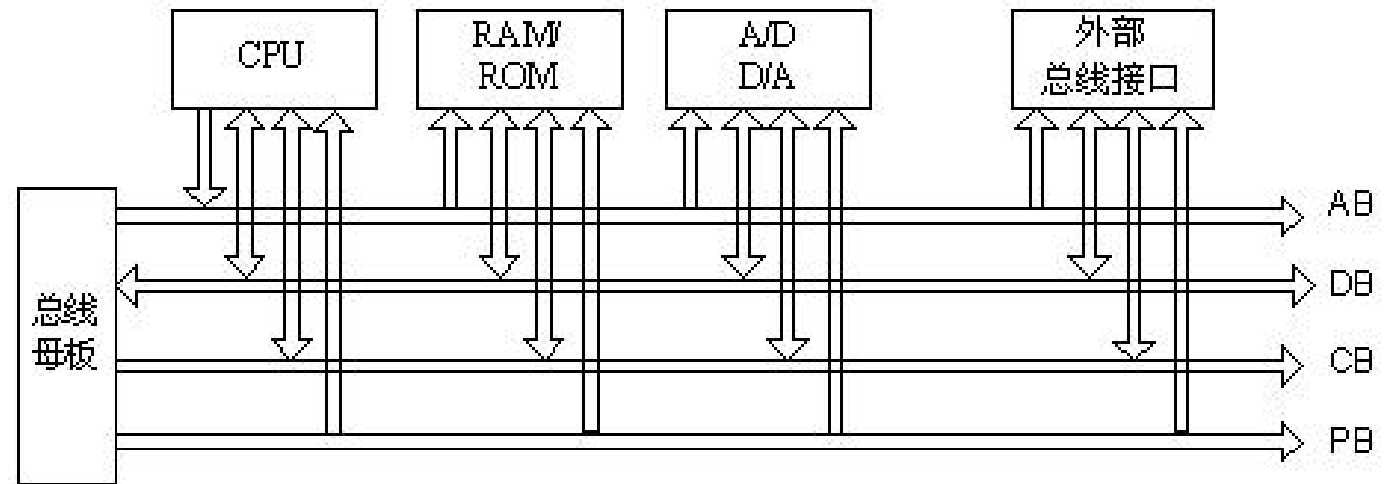
8.4 总线

第 97 页

- 在计算机系统中，各子系统之间可以通过总线互相连接。
- 总线标准——芯片之间、插板之间及系统之间，通过总线进行连接和传输信息时，应遵守的一些协议与规范

- 数据信息
- 控制信息
- 状态信息

- 优点：成本低、简单



- 主要缺点：它是由不同的外设分时共享的，形成了信息交换的瓶颈，从而限制了系统中总的I/O吞吐量。

8.4.1 总线的设计

1. 设计总线时需要考虑的一些问题

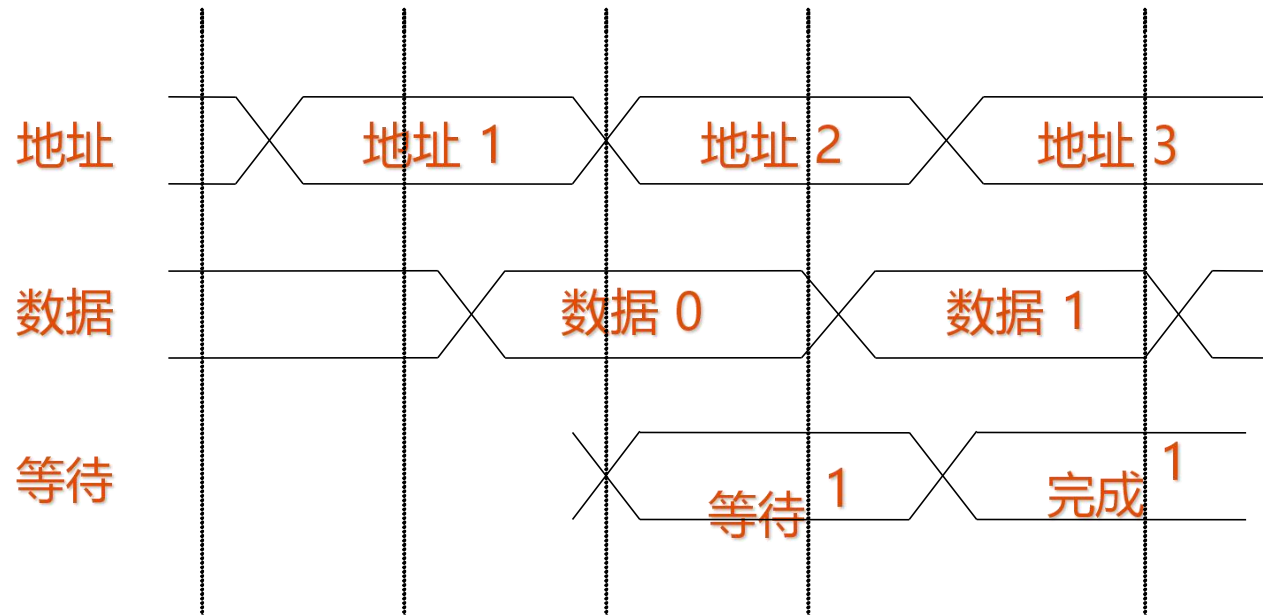
特性	高性能	低价格
总线宽度	独立的地址和数据总线	数据和地址分时 共用同一套总线
数据总线宽度	越宽越快（例如：64位）	越窄越便宜（例如：8位）
传输块大小	块越大总线开销越小	单字传送更简单
总线主设备	多个（需要仲裁）	单个（无需仲裁）
分离事务	采用——分离的请求包和回答包能提高总线带宽	不采用——持续连接成本更低，而且延迟更小
定时方式	同步	异步

8.4.1 总线的设计

第 99 页

3. 分离事务总线（又称：流水总线、悬挂总线、包交换总线）

- 在有多个主设备时，可以通过包交换来提高总线带宽，基本思想
 - 将总线事务分成请求和应答两部分。
 - 在请求和应答之间的空闲时间内，总线可以供其他的I/O使用，这样就不必在整个I/O过程中都独占总线。



分离事务总线有较高的带宽，但是它的数据传送延迟通常比独占总线方法大

4. 同步总线

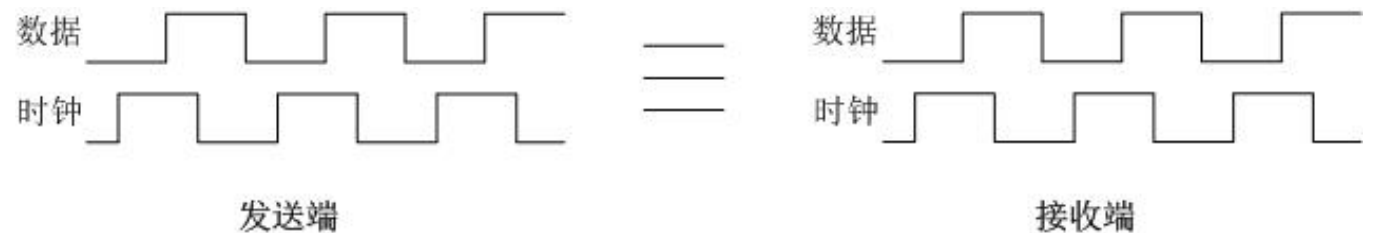
- 同步总线的控制线中包含一个时钟，总线上所有设备的所有的通信操作都以该时钟为基准。

- 优点：速度快、成本低。

- 缺点

- 由于时钟通过长距离传输后会扭曲，因而同步总线不能用于长距离的连接。特别是对于高速同步总线来说，更是如此。
- 总线上的所有设备都必须以同样的时钟频率工作。

- CPU-存储器总线通常是采用同步总线。



5. 异步总线

- 没有统一的参考时钟，每个设备都有各自的定时方法。
- 采用握手协议。
- 不存在时钟扭曲和同步的问题，传输距离可以比较长。
- 很多I/O总线都采用异步总线。
- 同步总线通常比异步总线快。



8.4.2 总线标准和实例

- 1. I/O总线标准：定义如何将设备与计算机进行连接的文档。
- 2. 常见I/O总线的一些典型特征

几种常用并行I/O总线

	IDE / Ultra ATA	SCSI	PCI	PCI-X
数据宽度 (b)	16	8/16	32/64	32/64
时钟频率 (MHz)	最高100	10 (Fast) 20 (Ultra) 40 (Ultra2) 80 (Ultra3) 160 (Ultra4)	33/66	66/100/133
总线主设备数量	1个	多个	多个	多个
峰值带宽 (MBps)	200	320	533	1066
同步方式	异步	异步	同步	同步
标准	无	ANSI X3.131	无	无

8.4.2 总线标准和实例

在嵌入式系统中使用较多的4种串行I/O总线的一些典型特征

	I ² C	1-wire	RS-232	SPI
数据宽度 (b)	1	1	2	1
信号线数量	2	1	9/25	3
时钟频率 (MHz)	0.4 ~ 10	异步	0.04或异步	异步
总线主设备数量	多个	多个	多个	多个
峰值带宽 (Mbps)	0.4 ~ 3.4	0.014	0.192	1
同步方式	异步	异步	异步	异步
标准	无	无	EIA, ITU-T V.21	无

8.4.2 总线标准和实例

在服务器系统中使用的CPU-存储器互连系统

	HP HyperPlane Crossbar	IBM SP	SUN Gigaplane-XB
数据宽度 (b)	64	128	128
时钟频率 (MHz)	120	111	83.3
总线的主设备数	多个	多个	多个
每端口峰值带宽 (MBps)	960	1700	1300
总峰值带宽 (MBps)	7680	14200	10667
同步方式	同步	同步	同步
标准	无	无	无

8.4.3 与CPU的连接

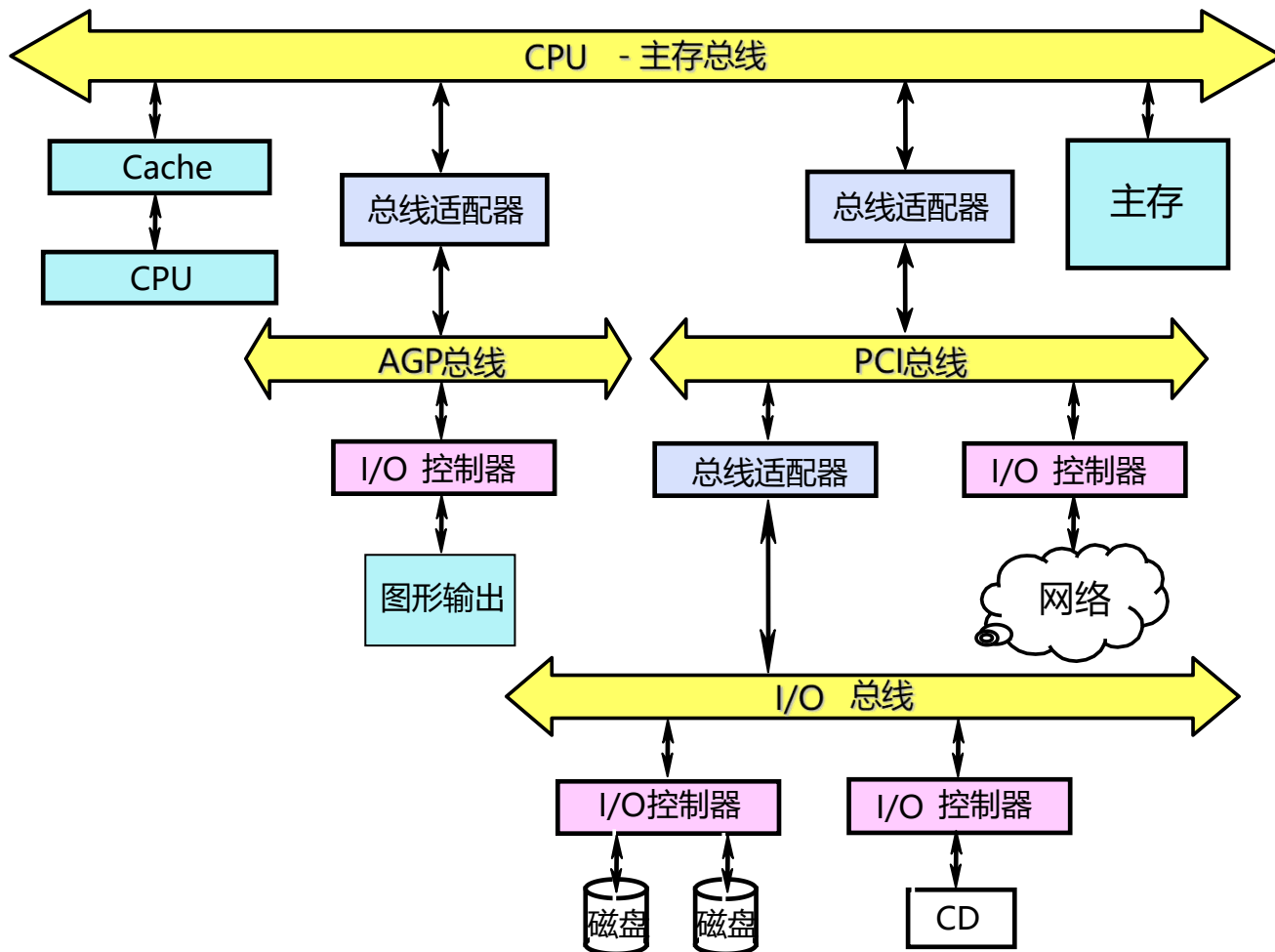
第 105 页

1. I/O总线的物理连接方式有两种选择

- 连接到存储器上（更常见）
- 连接到Cache上

2. I/O总线连接到存储器总线上的方式

- 一种典型的组织结构



3. CPU对I/O设备的编址有两种方式

- 存储器映射I/O（也称为I/O设备统一编址）
 - 将一部分存储器地址空间分配给I/O设备，用load指令和store指令对这些地址进行读写将引起I/O设备的数据传输。
 - 将一部分存储空间留出用于设备控制，对这一部分地址空间进行读写就是向设备发出控制命令。
- 给I/O设备独立编址
 - 需要在CPU中设置专用的I/O指令来访问I/O设备。
 - CPU需要发出一个标志信号来表示所访问的地址是I/O设备的地址。

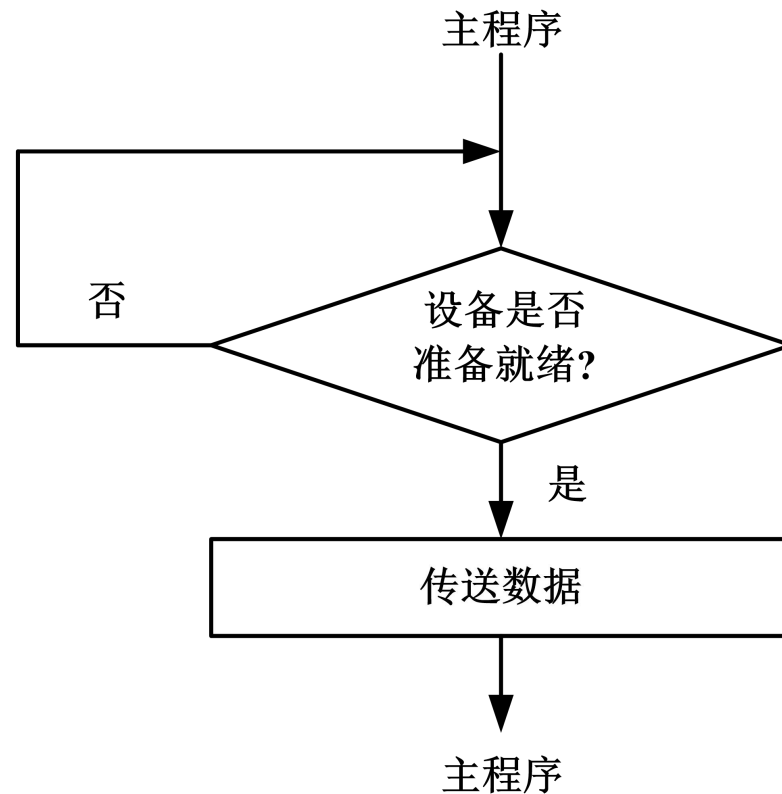
8.4.3 与CPU的连接

第 107 页

4. CPU与外部设备进行输入/输出的方式可分为4种

■ 程序查询

- 中断
- DMA
- 通道



查询方式:

- 优点: 方式简单
- 缺点:
 - 系统效率低
 - 响应速度慢

8.4.3 与CPU的连接

第 108 页

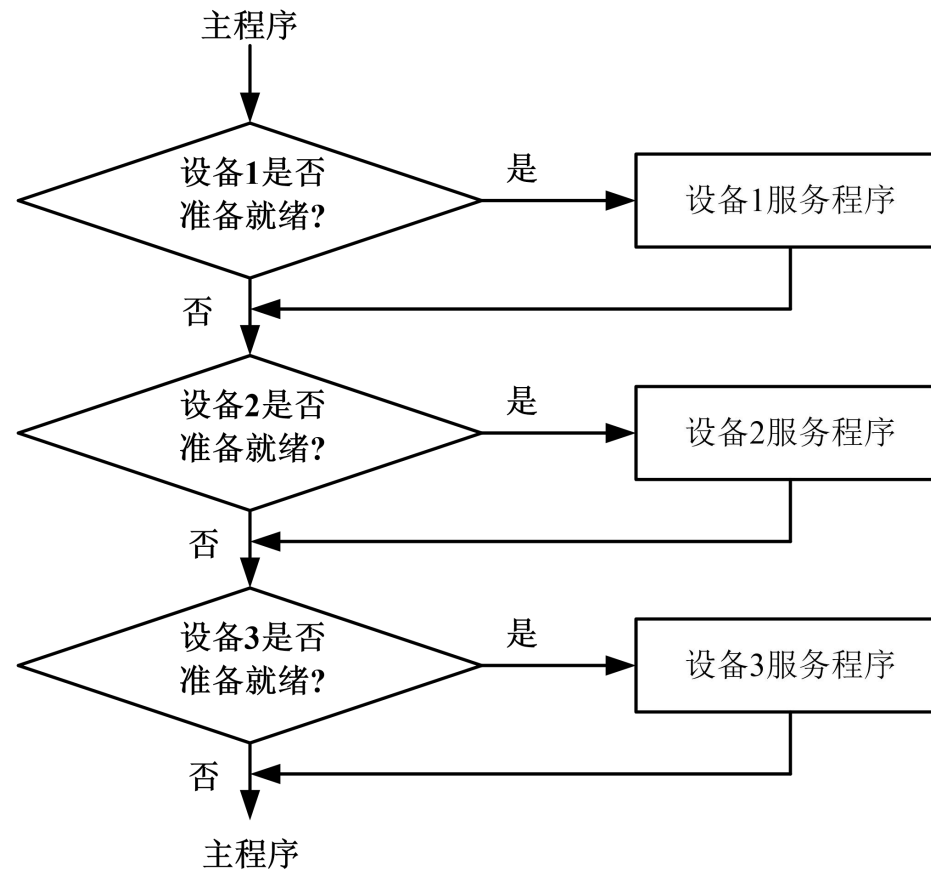
4. CPU与外部设备进行输入/输出的方式可分为4种

■ 程序查询

■ 中断

■ DMA

■ 通道



查询方式:

■ 优点: 方式简单

■ 缺点:

- 系统效率低
- 响应速度慢

8.4.3 与CPU的连接

第 109 页

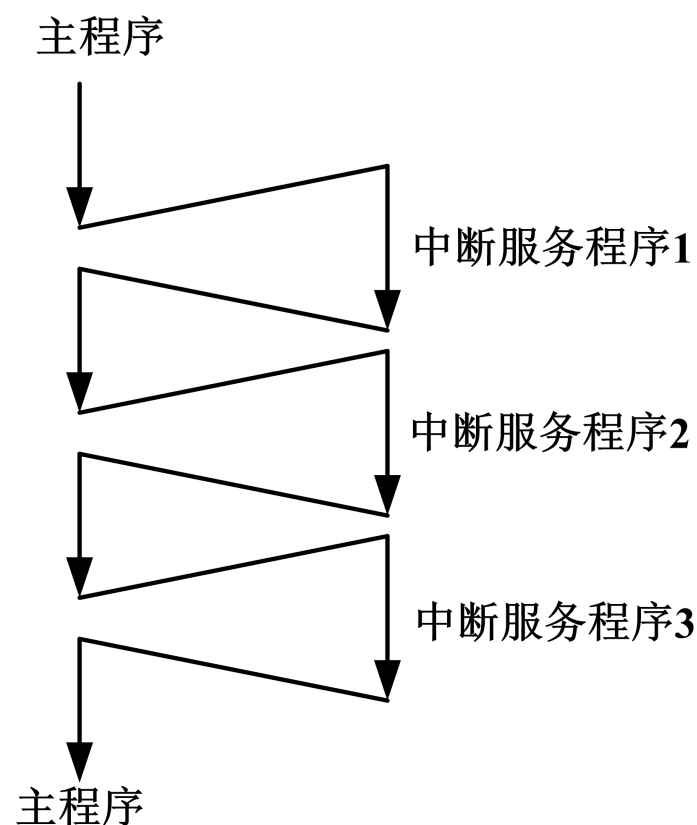
4. CPU与外部设备进行输入/输出的方式可分为4种

■ 程序查询

■ 中断

■ DMA

■ 通道



中断方式:

■ 外设具有申请处理器服务的主动权, 当输入设备准备好发送数据或输出设备准备好接收数据时, 便可以向处理器发出中断请求, 使处理器暂时停止目前正在运行的工作, 转而执行与外设进行数据传输的任务, 并在数据传输任务完成后处理器继续进行原来的工作。

4. CPU与外部设备进行输入/输出的方式可分为4种

- 程序查询
 - 中断
 - **DMA**
 - 通道
- DMA方式是一种完全由硬件执行数据传输的工作方式。在DMA方式下，外设利用专用的DMA控制器直接与存储器进行数据传送，占用很少的处理器资源，而且数据传输不必进行保护现场等额外操作，数据传输的速度基本上决定于外设和存储器的速度。
 - DMA方式传送数据则由DMA控制器来提供源和目的地址、修改地址、控制结束及发出控制信号。

4. CPU与外部设备进行输入/输出的方式可分为4种

- 程序查询
- 中断
- DMA
- 通道
- 优点：采用DMA方式，数据传送的速度显著提高，而且处理器的负担也明显减轻了，同时也缩短了数据传送的响应时间
- 缺点：DMA方式不适合小批量的数据传送
- 适用范围：
 - ◆ 操作系统的启动
 - ◆ 多处理器系统中多任务块传送
 - ◆ 快速的数据采集
 - ◆ 高速I/O设备的操作

4. CPU与外部设备进行输入/输出的方式可分为4种

- 程序查询
- 中断
- DMA
- 通道



均需CPU参与管理

- 查询方式的效率比较低，响应时间慢，但这并不意味着查询方式没有用。系统启动过程中，由于中断方式和DMA方式的初始化工作没有完成，所以此时对外设的操作只能采用查询方式。
- 中断方式是最常见的一种设备访问方式，即使是DMA方式，也必须与中断方式相结合，才能更好地完成对外设的操作。
- DMA方式是效率最高的一种设备访问方式，但只有在大数据量传输需求下采用DMA方式才能充分发挥它的效率。

8. 5 通道处理机

8.5.1 通道的作用和功能

第 114 页

1. 程序控制、中断和DMA方式管理外围设备会引起两个问题：

- 所有外设的输入/输出工作均由CPU承担，CPU的计算工作经常被打断而去处理输入/输出的事务，不能充分发挥CPU的计算能力。
- 大型计算机系统的外设虽然很多，但同时工作的机会不是很多。

解决上述问题的方法：采用通道处理机

- 通道处理机能够负担外围设备的大部分I/O工作。
- 通道处理机（简称通道）：专门负责整个计算机系统的输入/输出工作。通道处理机只能执行有限的一组输入/输出指令。

8.5.1 通道的作用和功能

第 115 页

3. 一个典型的由CPU、通道、设备控制器、外设构成的4级层次结构的输入/输出系统。

4. 通道的功能

- 接收CPU发来的I/O指令，并根据指令要求选择指定的外设与通道相连接。

- 执行通道程序

从主存中逐条取出通道指令，对通道指令进行译码，并根据需要向被选中的设备控制器发出各种操作命令。

- 给出外设中要进行读/写操作的数据所在的地址

如磁盘存储器的柱面号、磁头号、扇区号等。

8.5.1 通道的作用和功能

第 116 页

- 给出主存缓冲区的首地址

该缓冲区存放从外设输入的数据或者将要输出到外设中去的数据。

- 控制外设与主存缓冲区之间的数据传送的长度

对传送的数据个数进行计数，并判断数据传送是否结束。

- 指定传送工作结束时要进行的操作

例如：将外设的中断请求及通道的中断请求送往CPU等。

- 检查外设的工作状态是否正常，并将该状态信息送往主存指定单元保存。

- 在数据传输过程中完成必要的格式变换

例如：把字拆分为字节，或者把字节装配成字等。

5. 通道的主要硬件

■ 寄存器

- 数据缓冲寄存器
- 主存地址计数器
- 传输字节数计数器
- 通道命令字寄存器
- 通道状态字寄存器

■ 控制逻辑

- 分时控制
- 地址分配
- 数据传送、装配和拆分等

8.5.1 通道的作用和功能

第 118 页

6. 通道对外设的控制通过输入/输出接口和设备控制器进行

- 通道与设备控制器之间一般采用标准的输入/输出接口来连接。
- 通道通过标准接口把操作命令送到设备控制器，设备控制器解释并执行这些通道命令，完成命令指定的操作。
- 设备控制器能够记录外设的状态，并把状态信息送往通道和CPU。

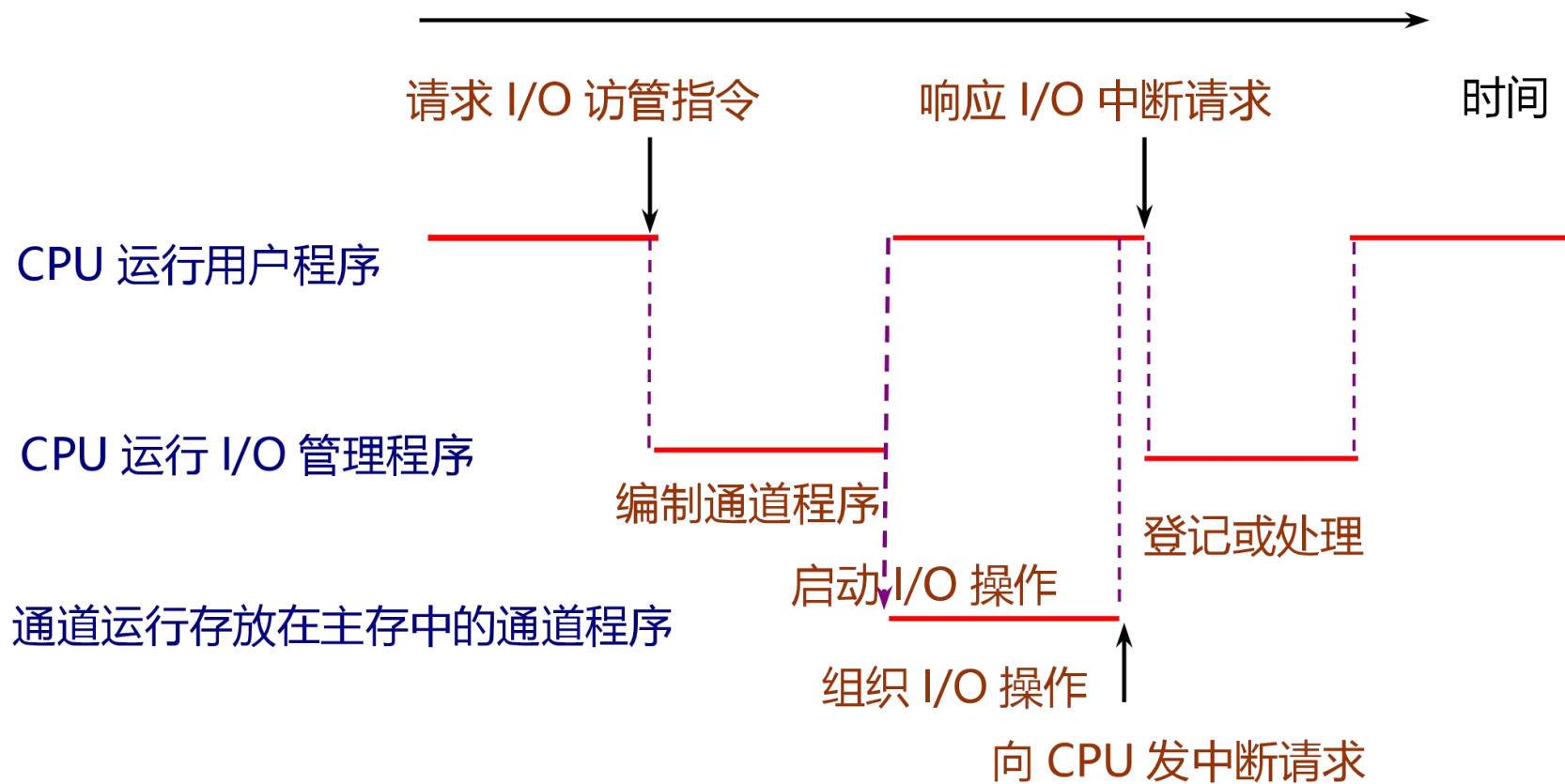
1. 通道完成一次数据输入/输出的工作过程

- 在用户程序中使用访管指令进入管理程序，由管理程序生成一个通道程序，并启动通道。
 - 用户在目标程序中设置一条广义指令，通过调用操作系统的管理程序来实现。
 - 管理程序根据广义指令提供的参数来编制通道程序。
 - 启动输入/输出设备指令是一条主要的输入/输出指令，属于特权指令。
- 通道处理机执行通道程序，完成指定的数据输入/输出工作。
 - 通道处理机执行通道程序与CPU执行用户程序是并行的。
- 通道程序结束后向CPU发中断请求。

8.5.2 通道的工作过程

第 120 页

2. CPU执行程序 and 通道执行程序的时间关系



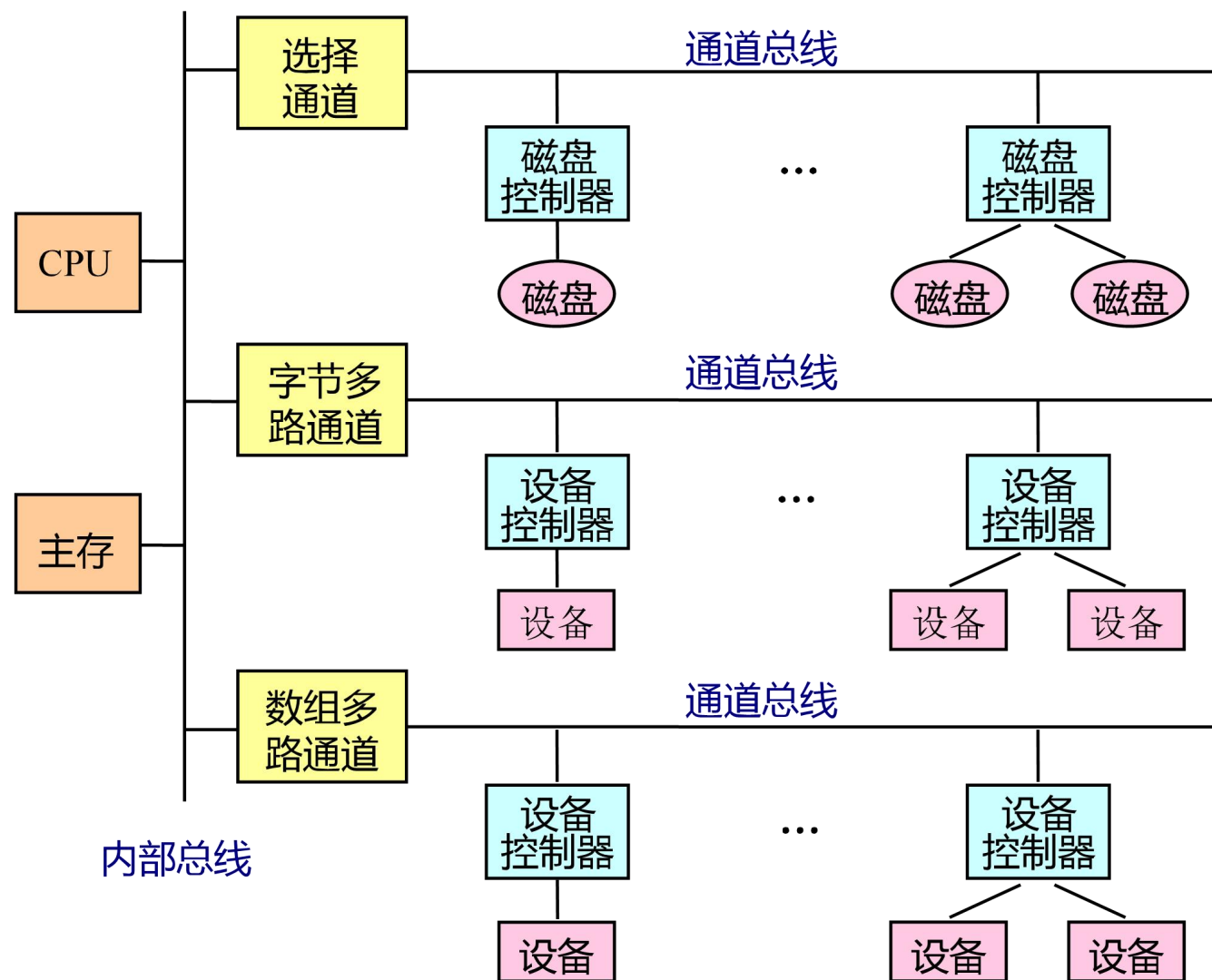
8.5.3 通道的种类

第 121 页

- 根据信息传送方式的不同，将通道分为三种类型

- 字节多路通道
- 选择通道
- 数组多路通道

- 三种类型的通道与CPU、设备控制器和外设的连接关系



1. 字节多路通道

- 为多台低速或中速的外设服务。
- 以字节交叉的方式分时轮流地为它们服务。
- 字节多路通道可以包含多个子通道，每个子通道连接一台设备控制器。

2. 选择通道

- 为多台高速外围设备服务；在一段时间内只为一台高速外设独占使用。
- 选择通道的硬件
 - 5个寄存器：数据缓冲寄存器、设备地址寄存器、主存地址计数器、交换字节数计数器、设备状态/控制寄存器
 - 格式变换部件：用于在主存和设备之间进行字与字节的拆分和装配
 - 通道控制部件

3. 数组多路通道

- 适用于高速设备。
- 每次选择一个高速设备后传送一个数据块，轮流为多台外围设备服务。
- 数组多路通道之所以能够并行地为多台高速设备服务，是因为虽然其所连设备的传输速率很高，但寻址等辅助操作时间很长。

以从磁盘存储器读出一个文件的过程为例：

- 数据读出过程可以分为3个步骤：磁头定位，找扇区和读出数据。
- 磁盘的寻址时间：磁头定位和找扇区的时间加起来
- 磁盘存储器的寻址时间要比数据传输时间长两个数量级以上。

8.5.4 通道流量分析

第 124 页

■ 通道流量

- 一个通道在数据传送期间，单位时间内能够传送的数据量。所用单位一般为B/s，又称为通道吞吐率、通道数据传输率等。
- 通道最大流量：一个通道在满负荷工作状态下的流量。

■ 参数的定义

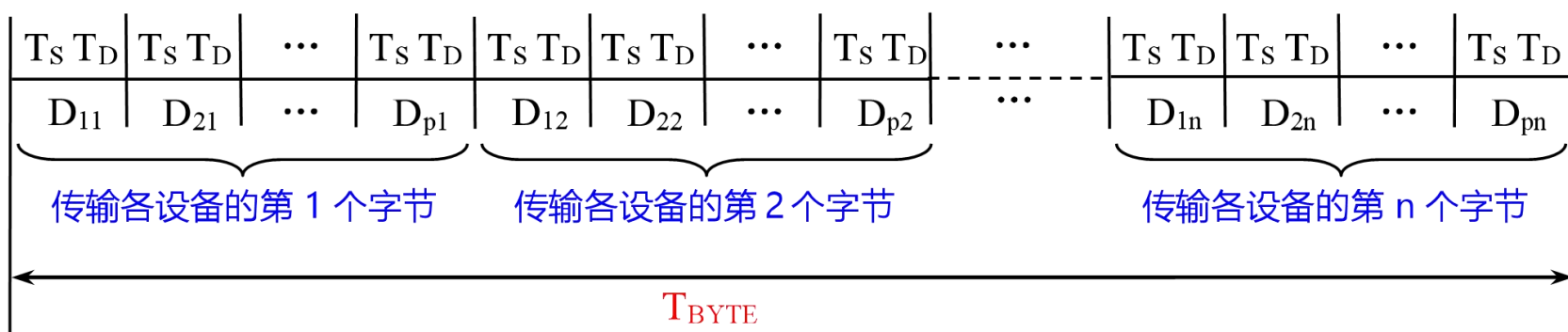
- T_s ：设备选择时间。从通道响应设备发出的数据传送请求开始，到通道实际为这台设备传送数据所需要的时间。
- T_D ：传送一个字节所用的时间。
- p ：在一个通道上连接的设备台数，且这些设备同时都在工作。
- n ：每台设备传送的字节数，这里假设每台设备传送的字节数都相同。
- k ：数组多路通道传输的一个数据块中包含的字节数。在一般情况下， $k < n$ 。对于磁盘、磁带等磁表面存储器，通常 $k=512$ 。
- T ：通道完成全部数据传送工作所需要的时间。

8.5.4 通道流量分析

第 125 页

1. 字节多路通道

■ 数据传送过程



- 通道每连接一台个外设，只传送一个字节，然后又与另一台设备连接，并传送一个字节。
- p台设备每台传送n个数据总共所需的时间为

$$T_{\text{BYTE}} = (T_S + T_D) \times p \times n$$

8.5.4 通道流量分析

第 126 页

1. 字节多路通道

- 最大流量

$$f_{\text{MAX-BYTE}} = \frac{pn}{(T_S + T_D)pn} = \frac{1}{T_S + T_D}$$

- 实际流量是连接在这个通道上的所有设备的数据传输率之和。

$$f_{\text{BYTE}} = \sum_{i=1}^p f_i \quad f_i: \text{第} i \text{台设备的实际数据传输率}$$

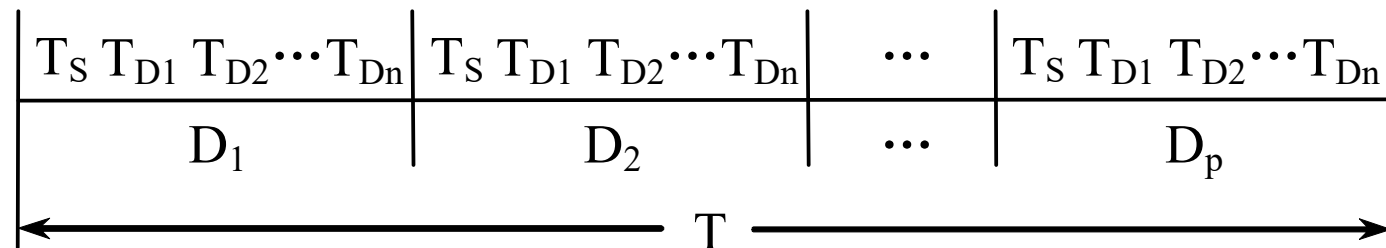
8.5.4 通道流量分析

第 127 页

2. 选择通道

- 在一段时间内只能单独为一台高速外设服务，当这台设备的数据传送工作全部完成后，通道才能为另一台设备服务。

- 工作过程



其中： D_i 表示通道正在为第*i*台设备服务， $T_{D1} = T_{D2} = \dots = T_{Dn} = T_D$

- p 台设备每台传送 n 个数据总共所需的时间

$$T_{SELECT} = (T_S + nT_D) \times P \quad \Rightarrow \quad T_{SELECT} = \left(\frac{T_S}{n} + T_D\right) \times p \times n$$

8.5.4 通道流量分析

第 128 页

■ 最大流量

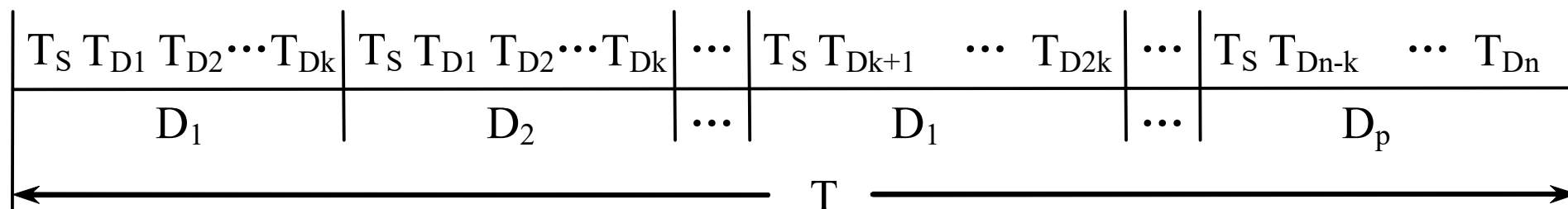
$$f_{\text{MAX-SELECT}} = \frac{pn}{(\frac{T_S}{n} + T_D)pn} = \frac{1}{\frac{T_S}{n} + T_D}$$

8.5.4 通道流量分析

第 129 页

3. 数组多路通道

■ 工作过程



■ p台设备每台传送n个数据总共所需的时间为：

$$T_{BLOCK} = \left(\frac{n}{k} T_S + n T_D \right) \times P \quad \Rightarrow \quad T_{BLOCK} = \left(\frac{T_S}{k} + T_D \right) \times p \times n$$

8.5.4 通道流量分析

第 130 页

■ 最大流量

$$f_{\text{MAX-BLOCK}} = \frac{pn}{(\frac{T_S}{k} + T_D)pn} = \frac{1}{\frac{T_S}{k} + T_D}$$

- 选择通道和数组多路通道的实际流量就是连接在这个通道上的所有设备中数据流量最大的那一个。

$$f_{\text{BLOCK}} \leq \max_{i=1}^p f_i \qquad f_{\text{SELECT}} \leq \max_{i=1}^p f_i$$

8.5.4 通道流量分析

第 131 页

- 各种通道的实际流量应该不大于通道的最大流量

$$f_{\text{BYTE}} \leq f_{\text{MAX-BYTE}}$$

$$f_{\text{BLOCK}} \leq f_{\text{MAX-BLOCK}}$$

$$f_{\text{SELECT}} \leq f_{\text{MAX-SELECT}}$$

- 两边的差值越小，通道的利用率就越高。
- 当两边相等时，通道处于满负荷工作状态。

8. 6 I/O与操作系统

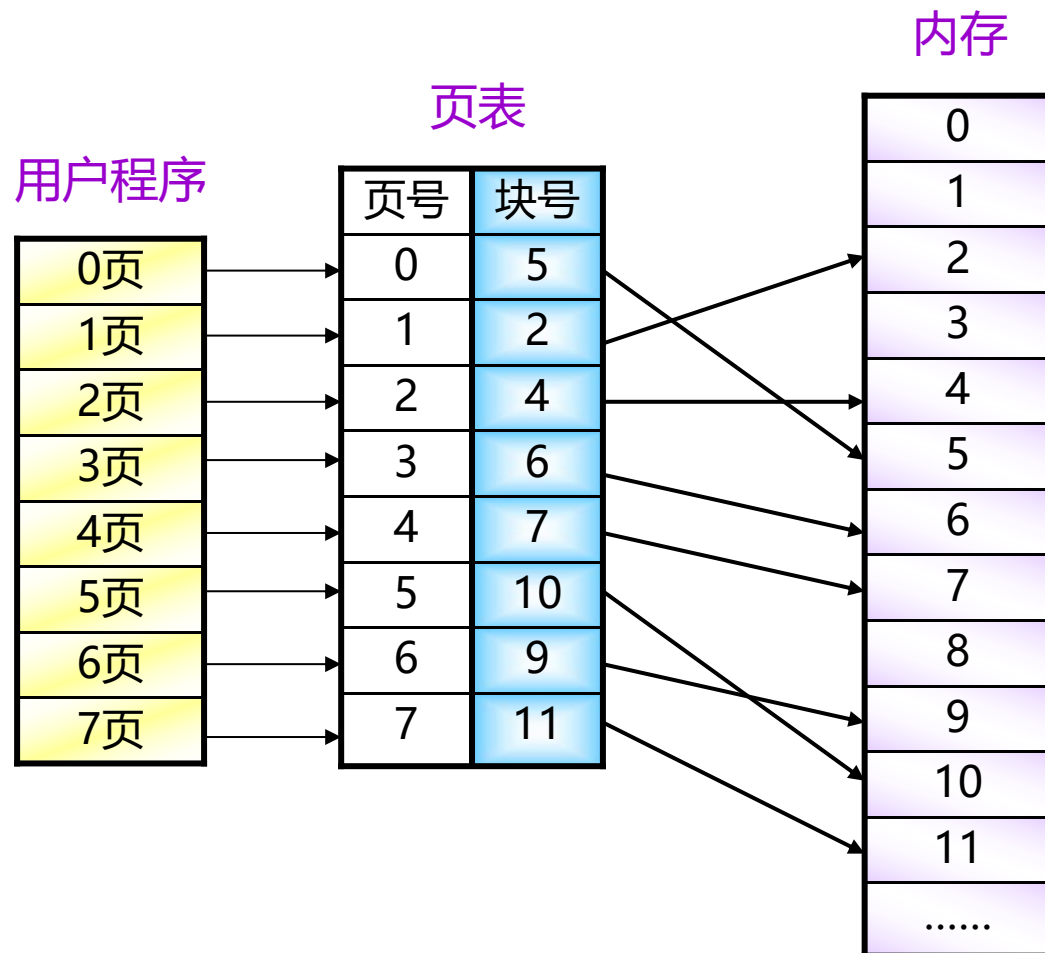
- 操作系统的作用之一是在多进程之间进行进程保护，这种保护包括存储器访问和I/O操作两个方面，主要存在两个方面的问题：
 1. DMA是使用虚拟地址还是物理地址？
 2. I/O和Cache数据是否一致？

8.6.1 DMA和虚拟存储器

第 134 页

■ 使用物理地址进行DMA传输，存在以下两个问题：

- 对于超过一页的数据缓冲区，由于缓冲区使用的页面在物理存储器中不一定是连续的，所以传输可能会发生问题。
- 如果DMA正在存储器和缓冲区之间传输数据时，操作系统从存储器中移出（或重定位）一些页面，那么，DMA将会在存储器中错误的物理页面上进行数据传输。

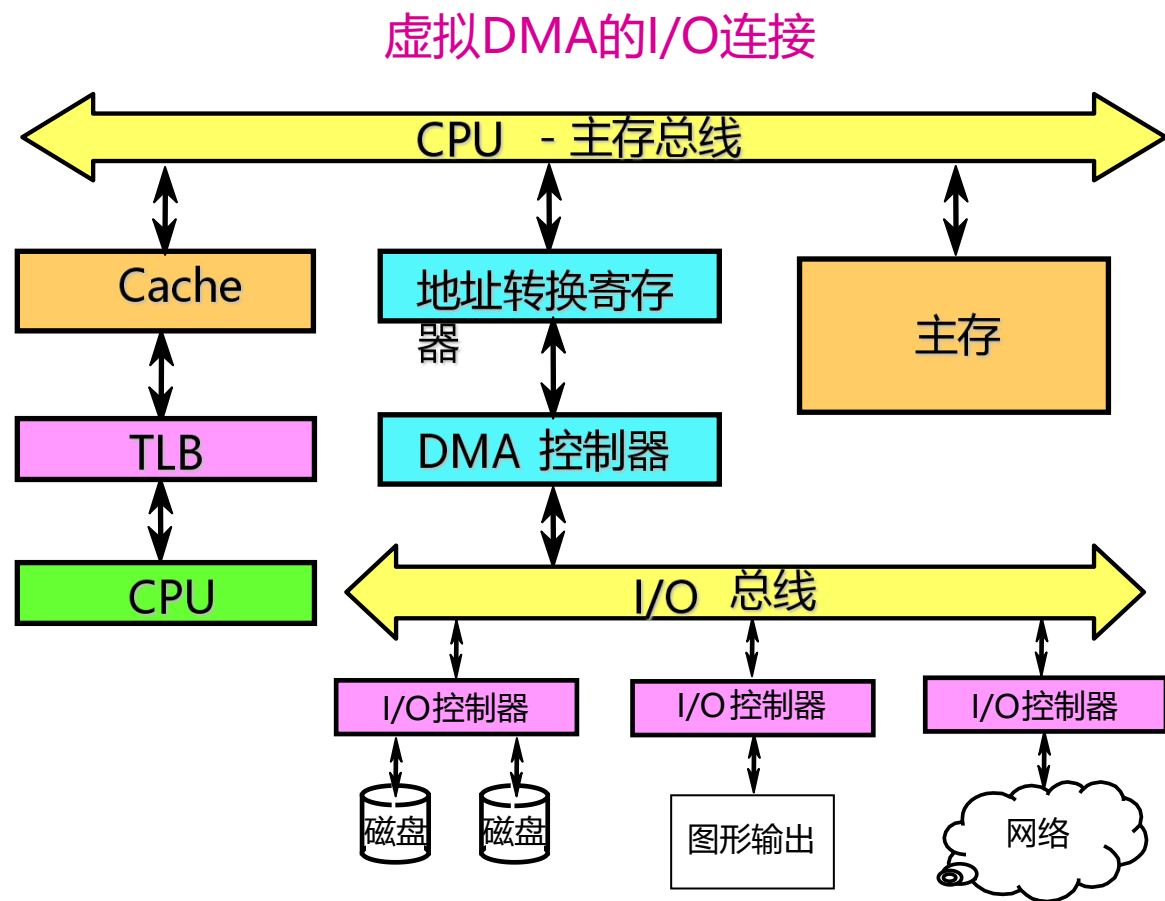


8.6.1 DMA和虚拟存储器

第 135 页

■ 解决这些问题的方法——“虚拟DMA”技术

- 允许DMA设备直接使用虚拟地址，并在DMA期间由硬件将虚拟地址转换为物理地址。
- 在采用虚拟DMA的情况下，如果进程在内存中被移动，操作系统应该能够及时地修改相应的DMA地址表。

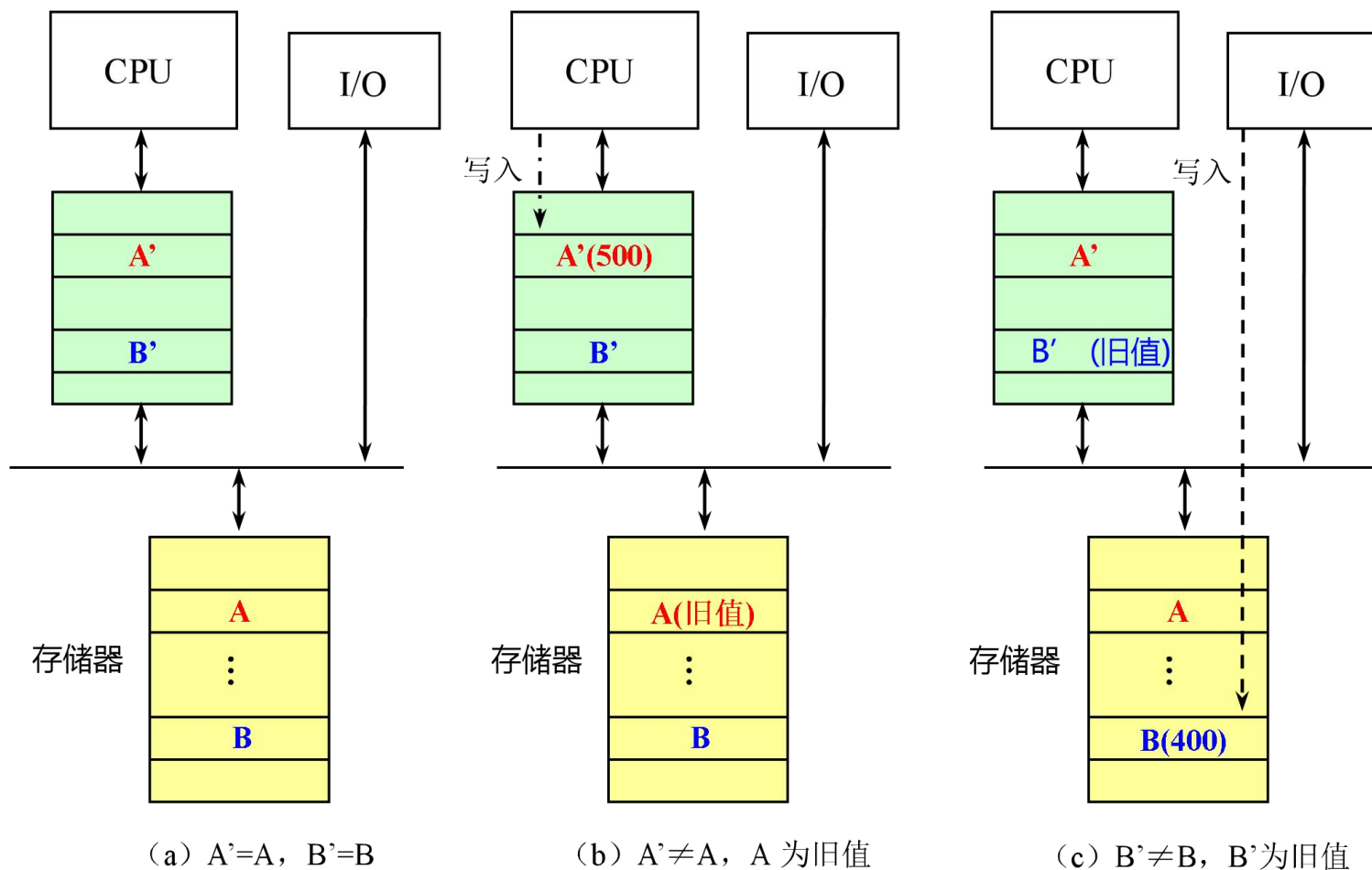


1. Cache会使一个数据出现两个副本：一个在Cache中，另一个在主存中。
2. I/O设备可以修改存储器中的内容
 - 把I/O连接到存储器上会出现以下情况：
 - CPU修改了Cache的内容后，由于存储器的内容跟不上Cache内容的变化，I/O系统进行输出操作时所看到的数据是旧值。（写直达Cache没有这样的问题）
 - I/O系统进行输入操作后，存储器的内容发生了变化，但CPU在Cache中所看到的内容依然是旧值。

8.6.2 I/O和Cache数据一致性

第 137 页

举例：假设Cache采用写回法，并且A' 是A的副本，B' 是B的副本。



■ 把I/O直接连接到Cache上

- 不会产生由I/O导致的数据不一致的问题。
 - ✓ 所有I/O设备和CPU都能在Cache中看到最新的数据。
- I/O会跟CPU竞争访问Cache，在进行I/O时，会造成CPU的停顿。
- I/O还可能会破坏Cache中CPU访问的内容，因为I/O操作可能导致一些新数据被加入Cache，而这些新数据可能在近期内并不会被CPU访问。

3. 解决内容一致性的方法

(不管Cache是采用写直达法还是写回法)

■ 软件的方法

- 设法保证I/O缓冲器中的所有各块都不在Cache中。
- 具体做法有两种
 - ✓ 把I/O缓冲器的页面设置为不可进入Cache的，在进行输入操作时，操作系统总是把输入的数据放到该页面上。
 - ✓ 在进行输入操作之前，操作系统先把Cache中与I/O缓冲器相关的数据“赶出”Cache，即把相应的数据块设置为“无效”状态。

3. 解决内容一致性问题的方法

■ 硬件的方法

- 在进行输入操作时，检查相应的I/O地址（I/O缓冲器中的单元）是否在Cache中（即是否有数据副本）。
- 如果发现I/O地址在Cache中有匹配的项，就把相应的Cache块设置为“无效”。

1. I/O系统的性能：存储系统IO系统、连接特性、容量、响应时间、吞吐率等
2. 总线：定义、分离事务总线、同步总线、异步总线、程序查询、中断、DMA等
3. 通道处理机：字节多路通道、通道选择、数组多路通道
4. I/O与操作系统：虚拟存储器、数据一致性

应用题

第 142 页

1、设某个数组多路通道设备选择时间 T_S 为 $1\mu s$ ，传送一个字节数据所需的时间 T_D 为 $1\mu s$ ，一次传送定工数据块的大小为512B。现有8台外设的数据传输速率分别如下表所示，问哪些外设可连接到该通道上能够正常工作。

设备名称	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	D ₈
数据传输速率	1000	480	480	800	512	512	1024	1024

解：数组多路通道的最大流量为

$$f_{max} = \frac{1}{\frac{T_S}{k} + T_D} = \frac{k}{T_S + k \cdot T_D} = \frac{512}{1 + 512 \times 1} = \frac{512}{513} (B/\mu s) < 1000KB/s$$

连接到通道上的外设能够正常工作的条件是访外设的数据传输速率 f_i 满足：

$$f_i < f_{max} < 1000KB/s$$

因此可能连接到通道上正常工作的外设为：D₂、D₃、D₄、D₅、D₆。

2、设某个字节多路通道的设备选择时间 T_S 为 $9.8\mu s$ ，传送一个字节的数据所需的时间 T_D 为 $0.2\mu s$ 。若某种低速外设每隔 $500\mu s$ 发出一次传送请求，则该通道最多可连接多少台这种外设？

解：字节多路通道的最大流量为：
$$f_{\text{MAX-BYTE}} = \frac{pn}{(T_S + T_D)pn} = \frac{1}{T_S + T_D}$$

字节多路通道的实际流量为：
$$f_{\text{BYTE}} = \sum_{i=1}^p f_i$$

其中， p 为通道连接的外设台数， f_i 为外设 i 的数据传输速率。由于连接的是同样的外设，所以 $f_1=f_2=\dots=f_p=f$ ，故有 $f_{\text{byte}}=pf$ 。

通道流量匹配的要求有： $f_{\text{max_byte}} \geq f_{\text{byte}}$

即有： $1/(T_S+T_D) \geq pf$ ；可得 $p \leq 1/(T_S+T_D)f$

已知 $T_S=9.8\mu s$ ， $T_D=0.2\mu s$ ， $1/f=500\mu s$ ，可求出通道最多可连接的设备台数为

$$p \leq 1/(T_S+T_D)f = 500\mu s / (9.8\mu s + 0.2\mu s) = 50$$

应用题

第 144 页

3、一个字节多路通道连接有6台设备，它们的数据传输率如下表所示。

设备名称	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆
数据传输速率 (B/ms)	50	50	40	25	25	10

(1) 计算该通道的实际工作量。

(2) 若通道的最大流量等于实际工作量，求通道的工作周期 T_S+T_D 。

解：(1) 通道实际流量为： $f_{\text{BYTE}} = \sum_{i=1}^p f_i = 50+50+40+25+25+10=200\text{B/ms}$

(2) 由于通道的最大流量等于实际工作流量，即有

$$f_{\text{MAX-BYTE}} = \frac{pn}{(T_S + T_D)pn} = \frac{1}{T_S + T_D} = 200\text{B/ms}$$

可得，通道的工作周期 $T_S+T_D=5\mu\text{s}$

第9章 互连网络